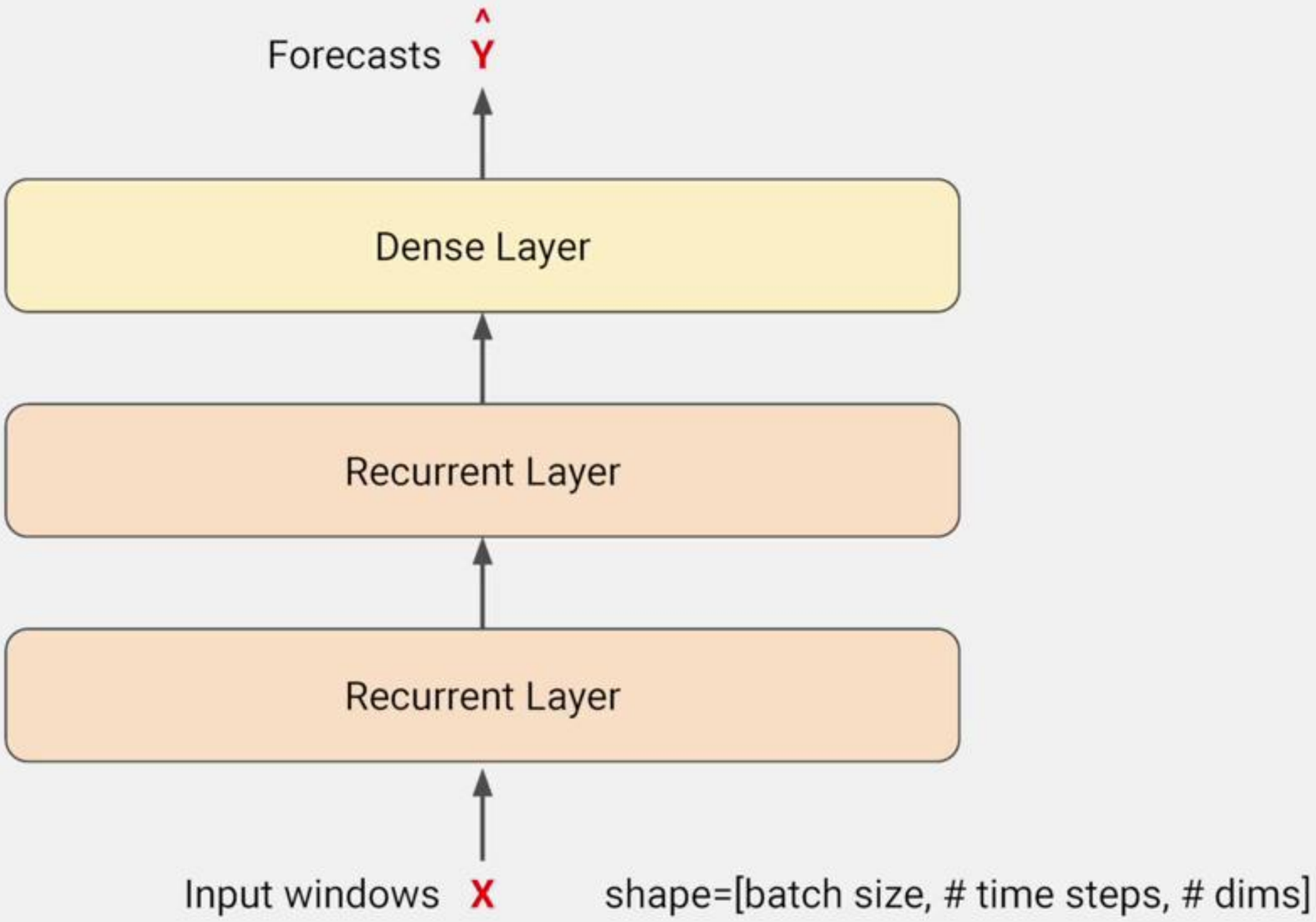
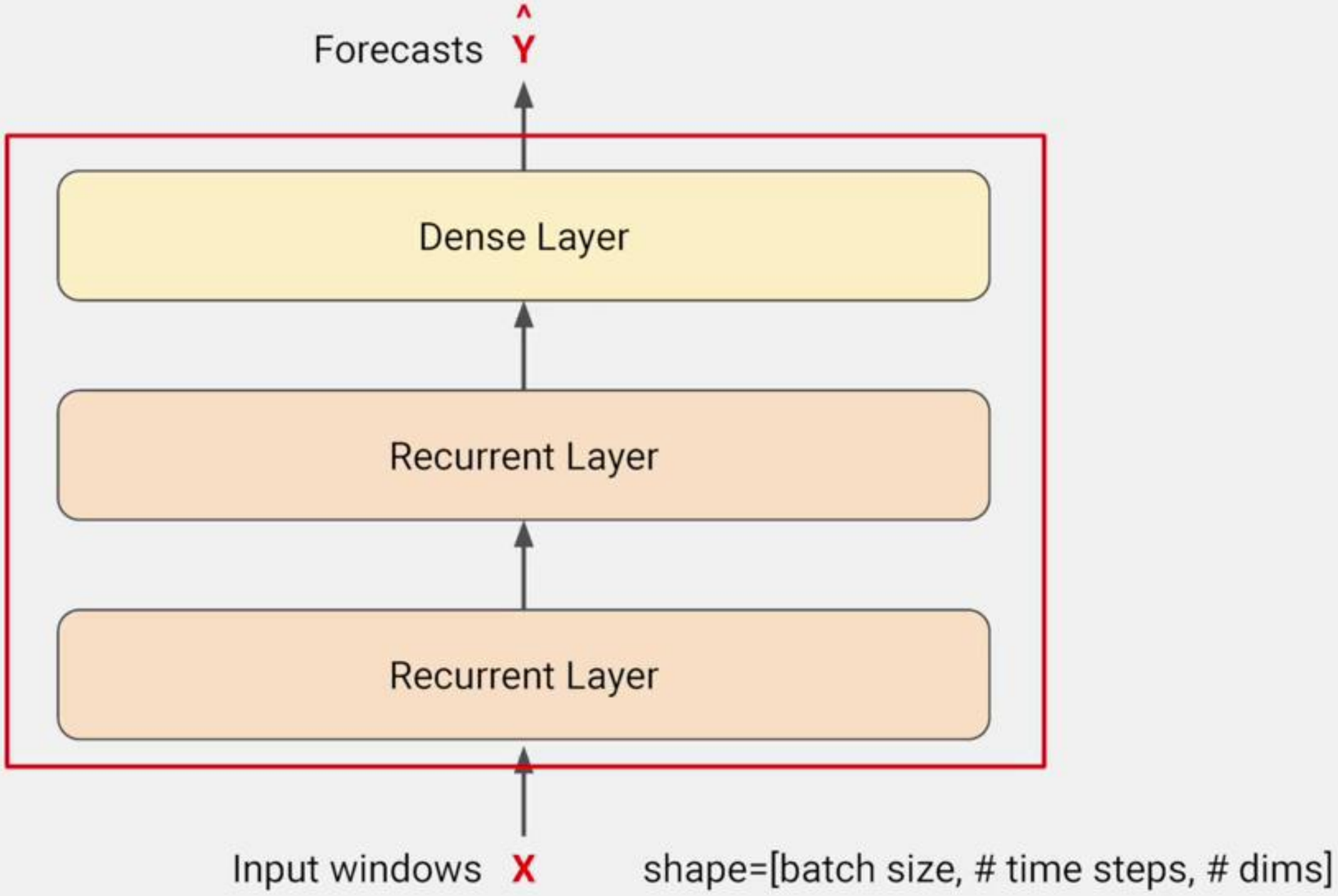
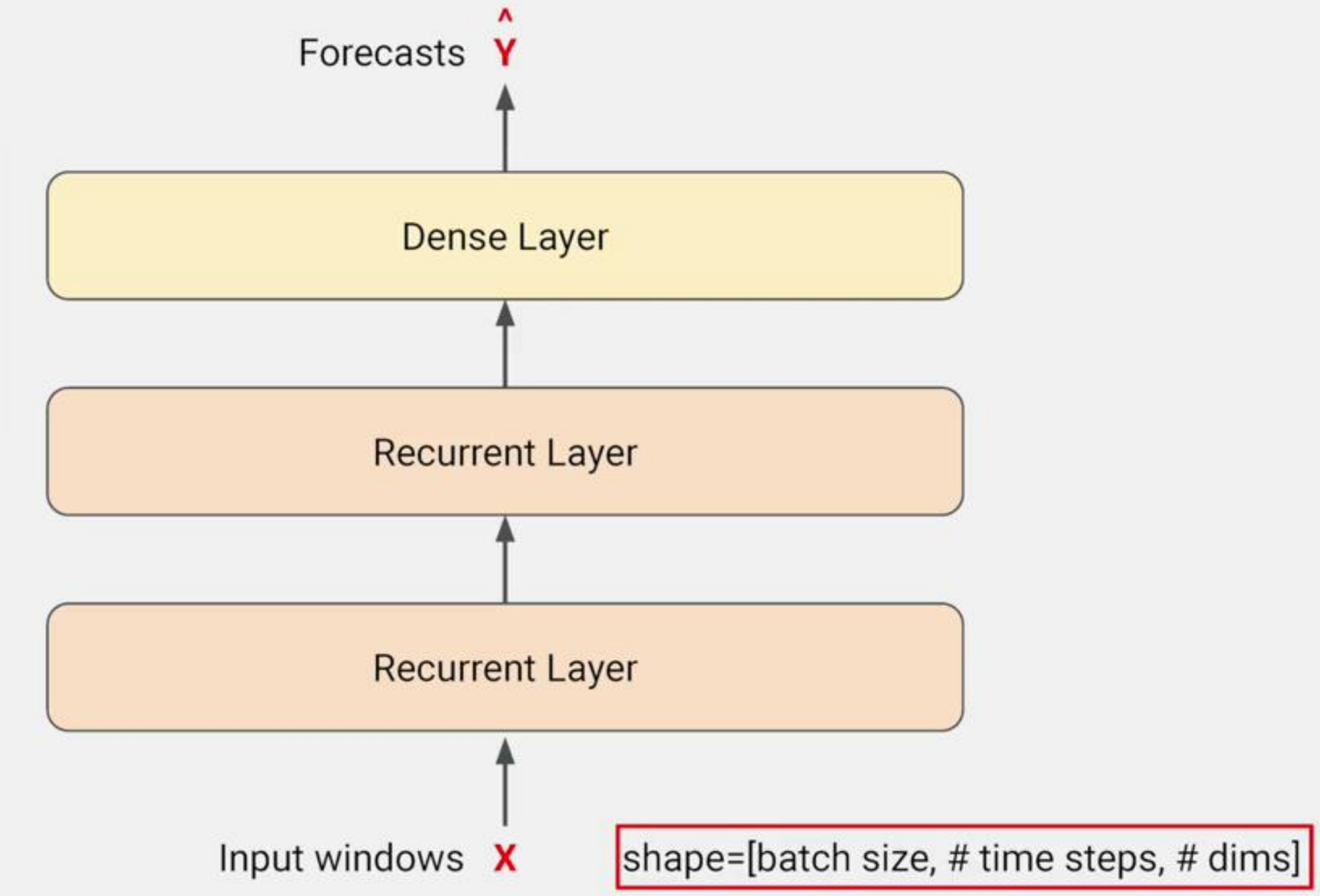
Recurrent Neural Network



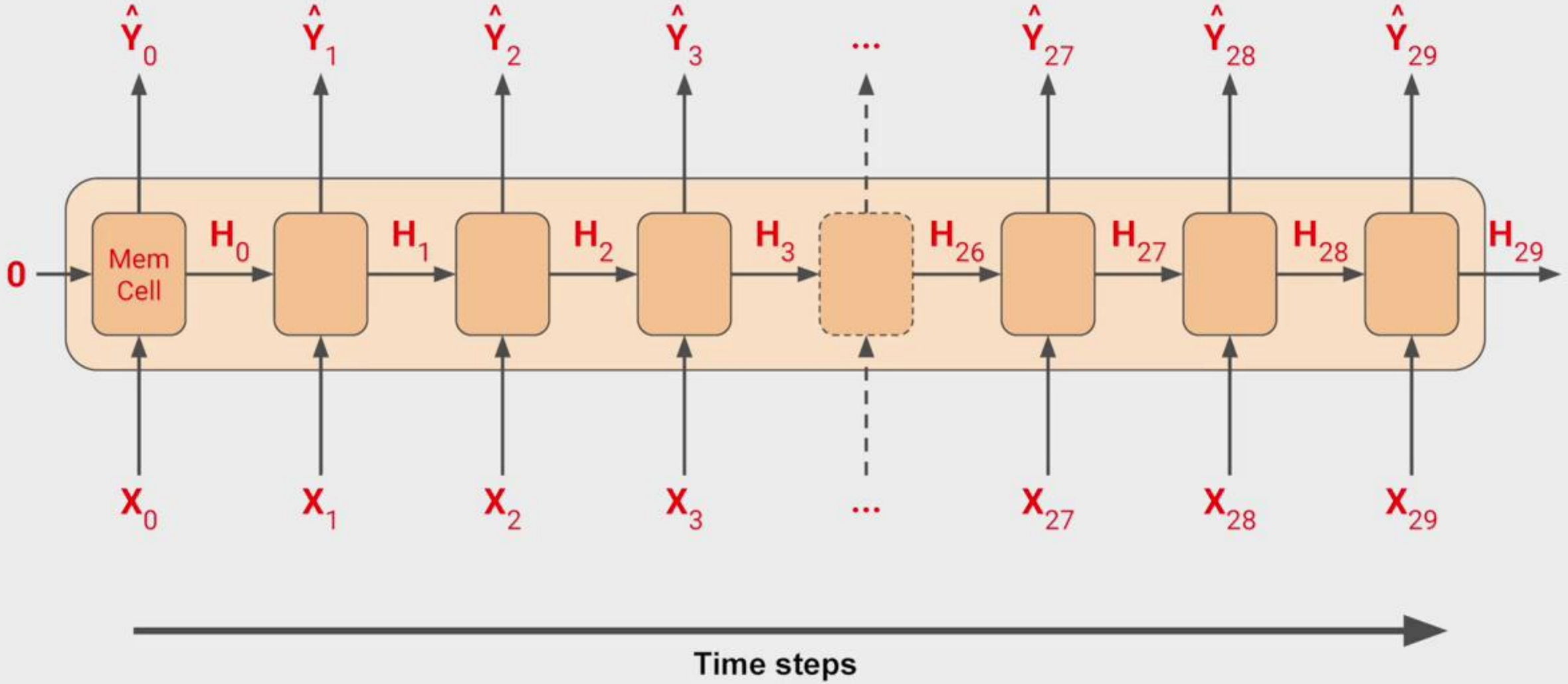
Recurrent Neural Network



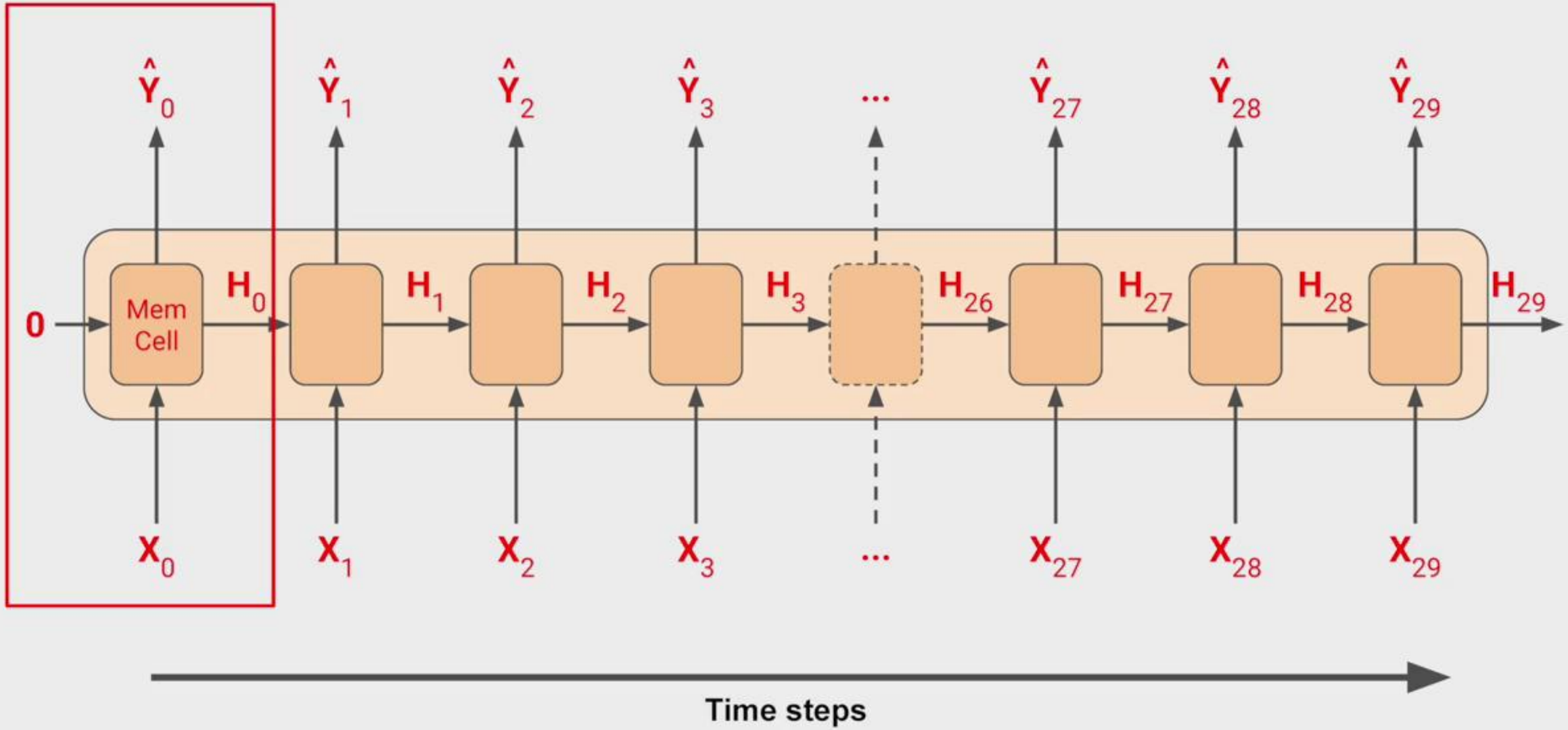
Recurrent Neural Network



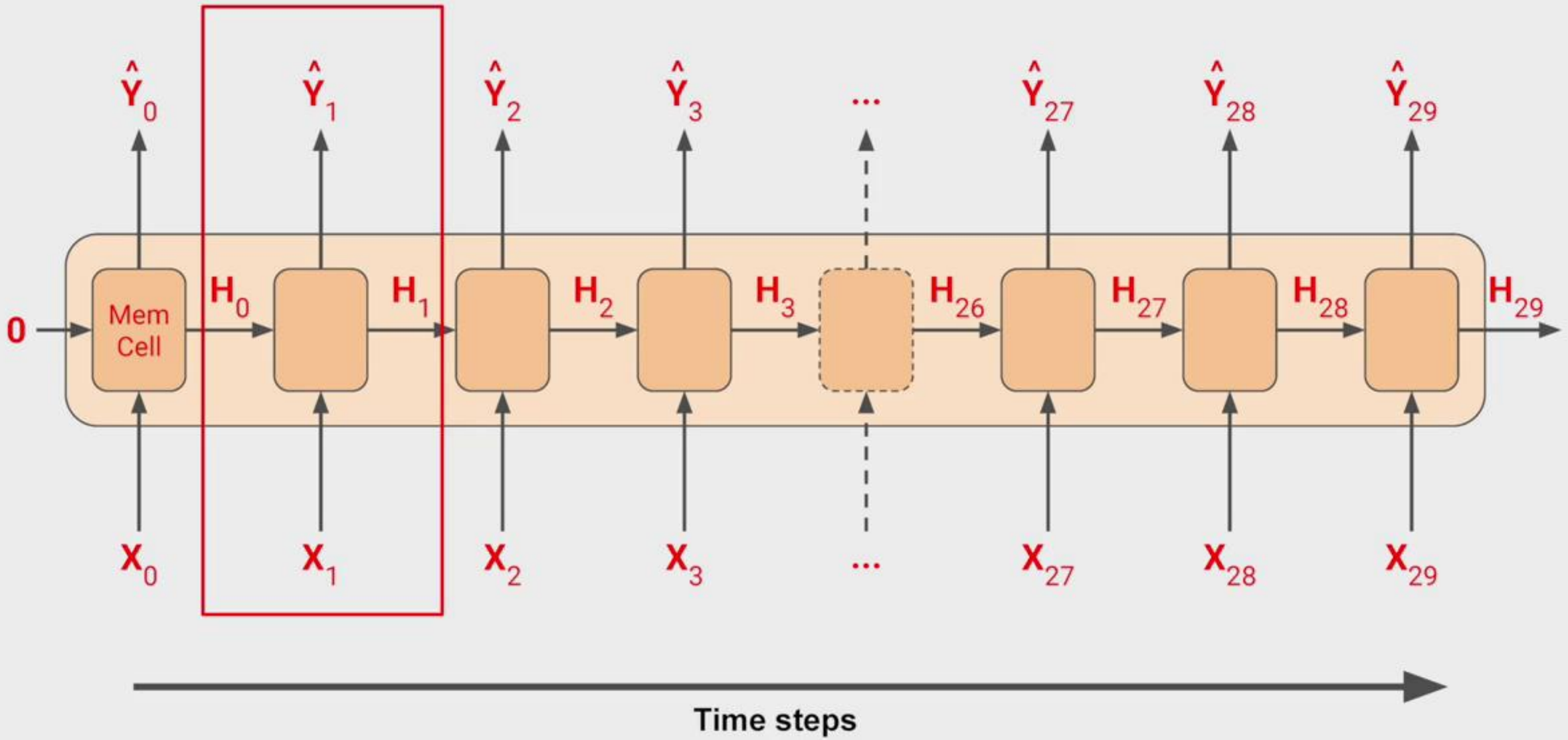
Recurrent Layer



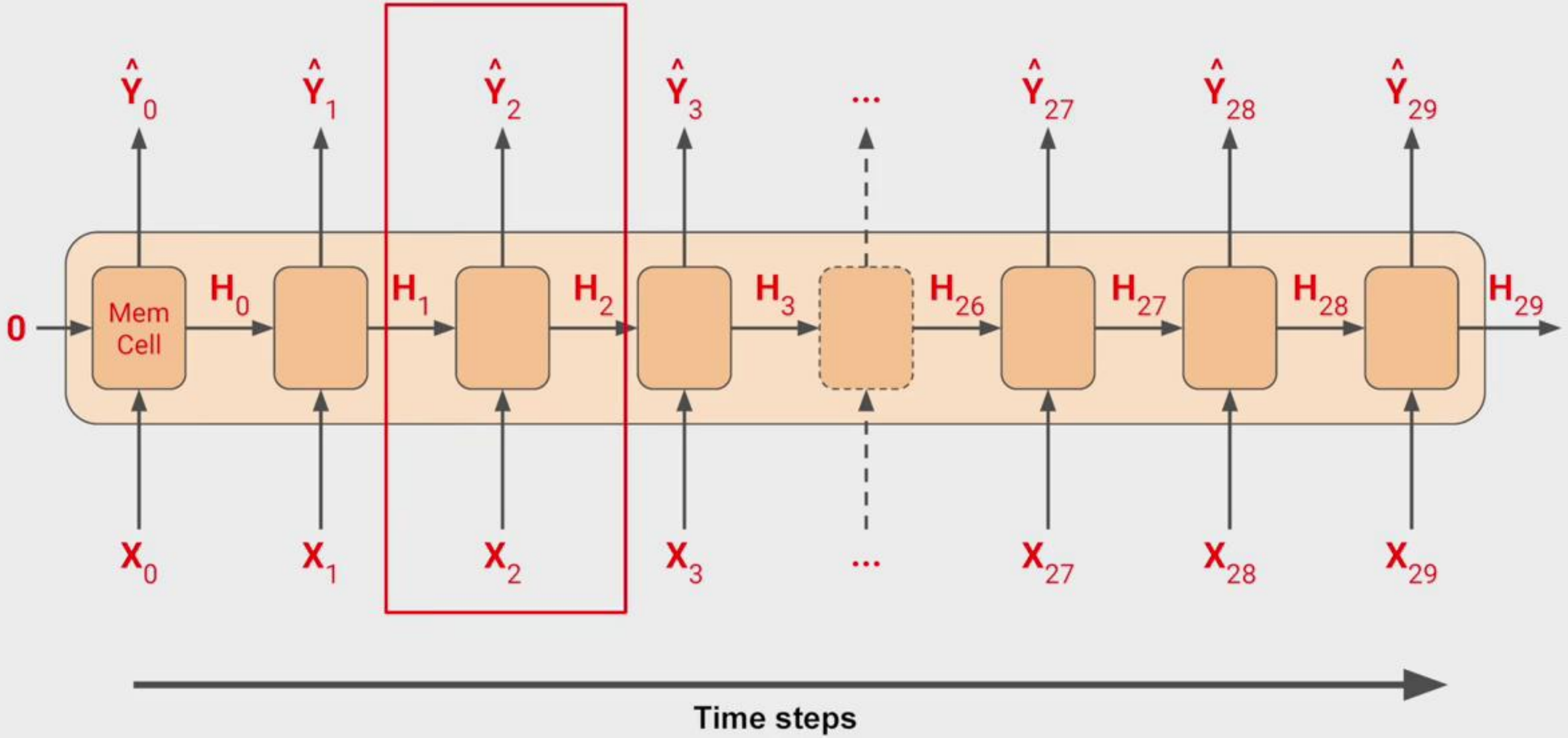
Recurrent Layer



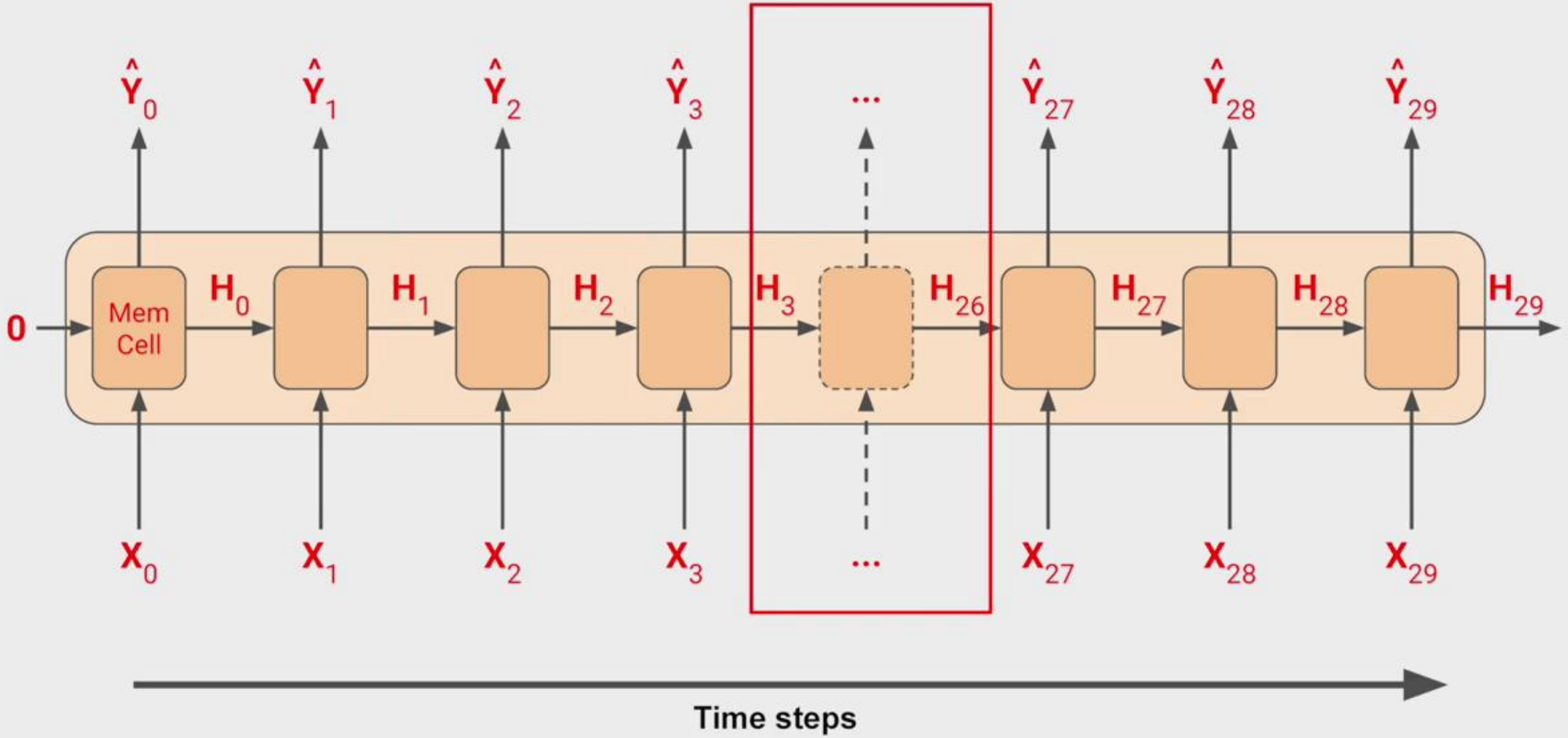
Recurrent Layer



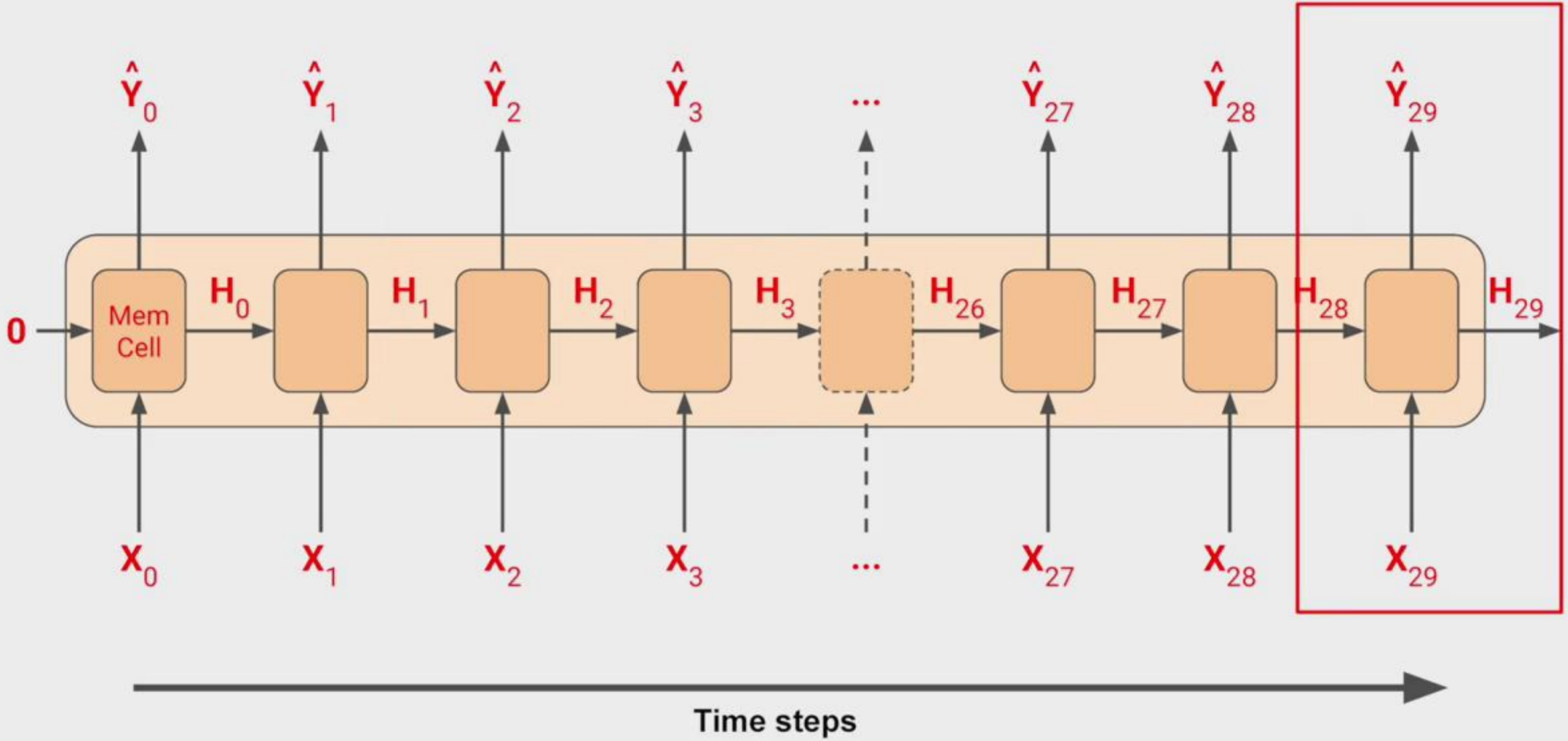
Recurrent Layer



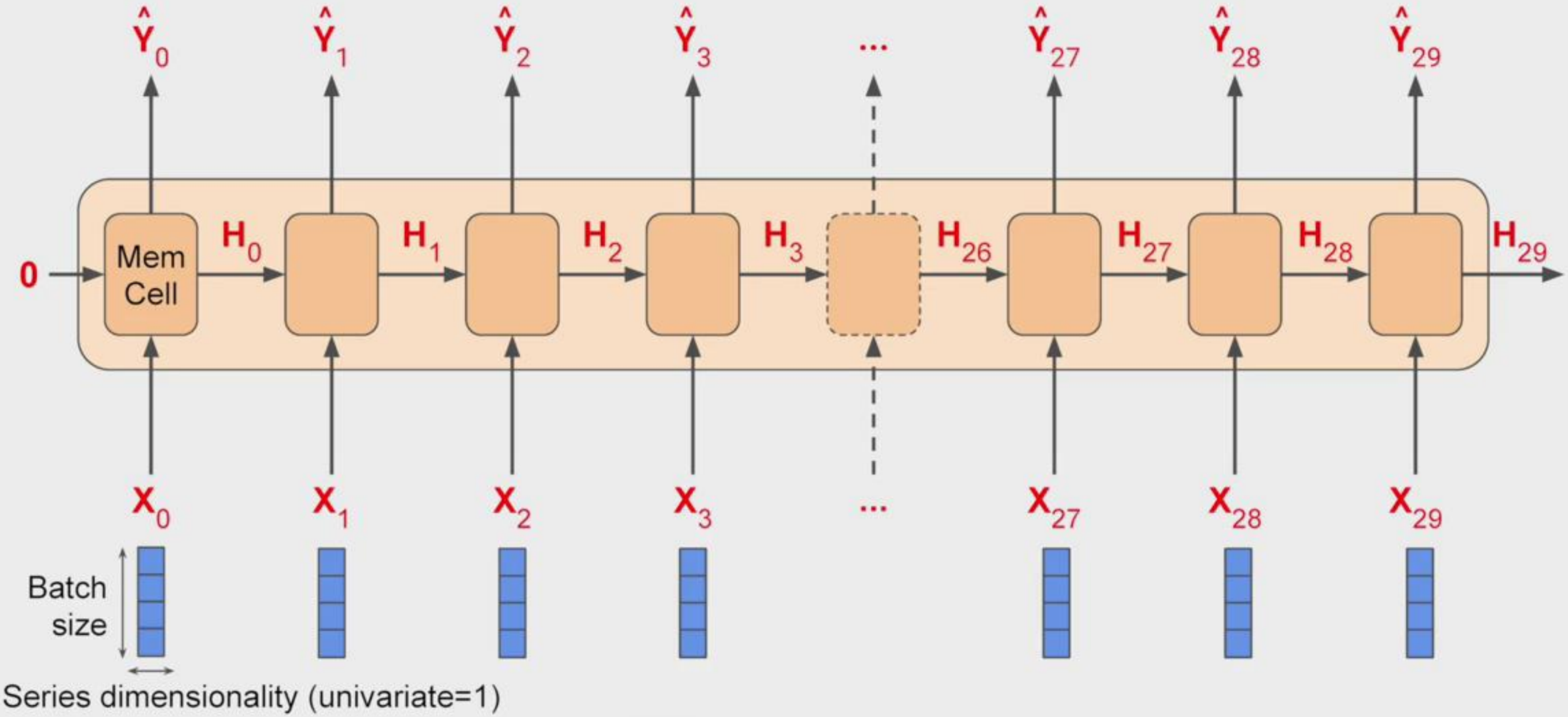
Recurrent Layer

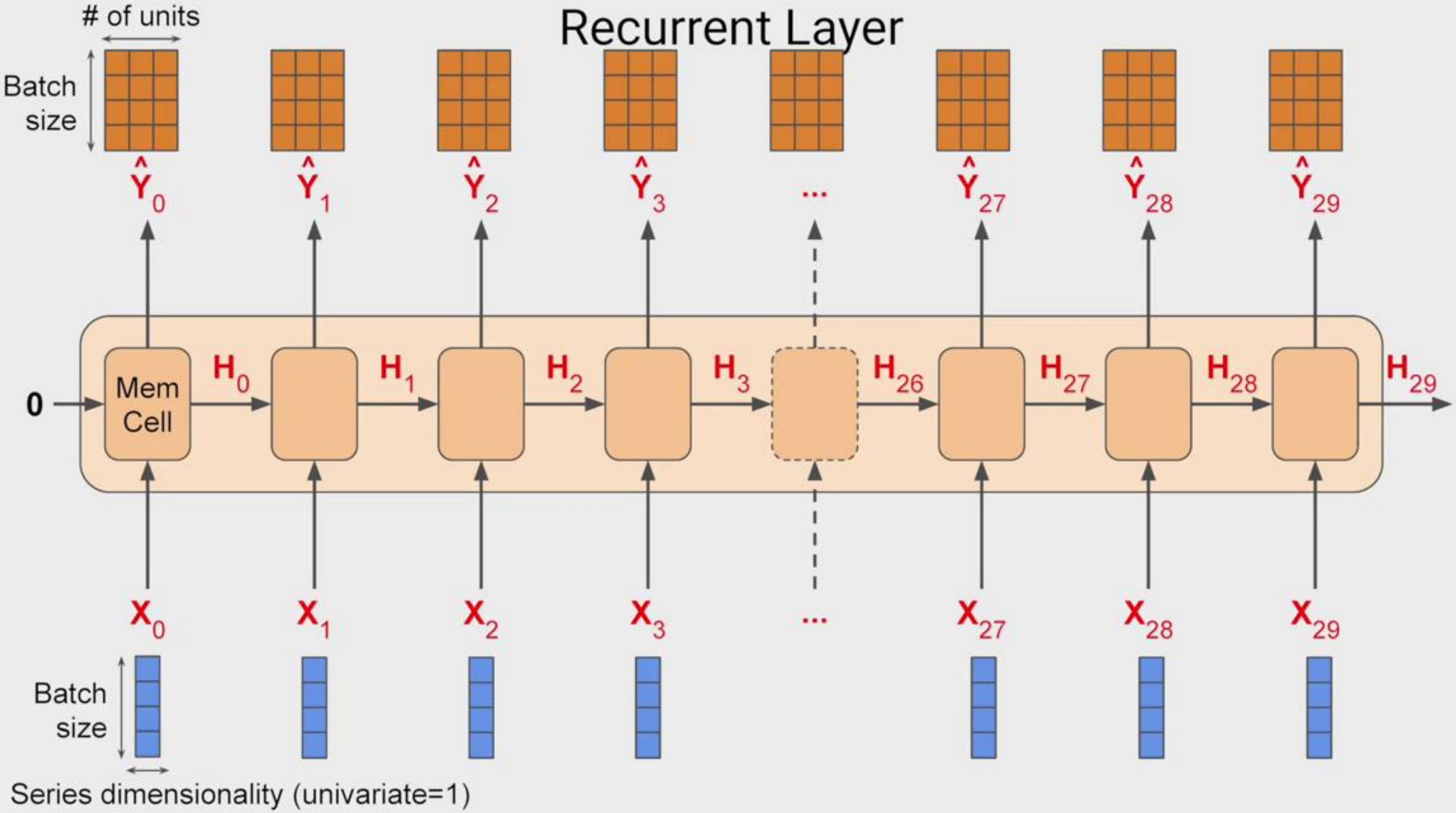


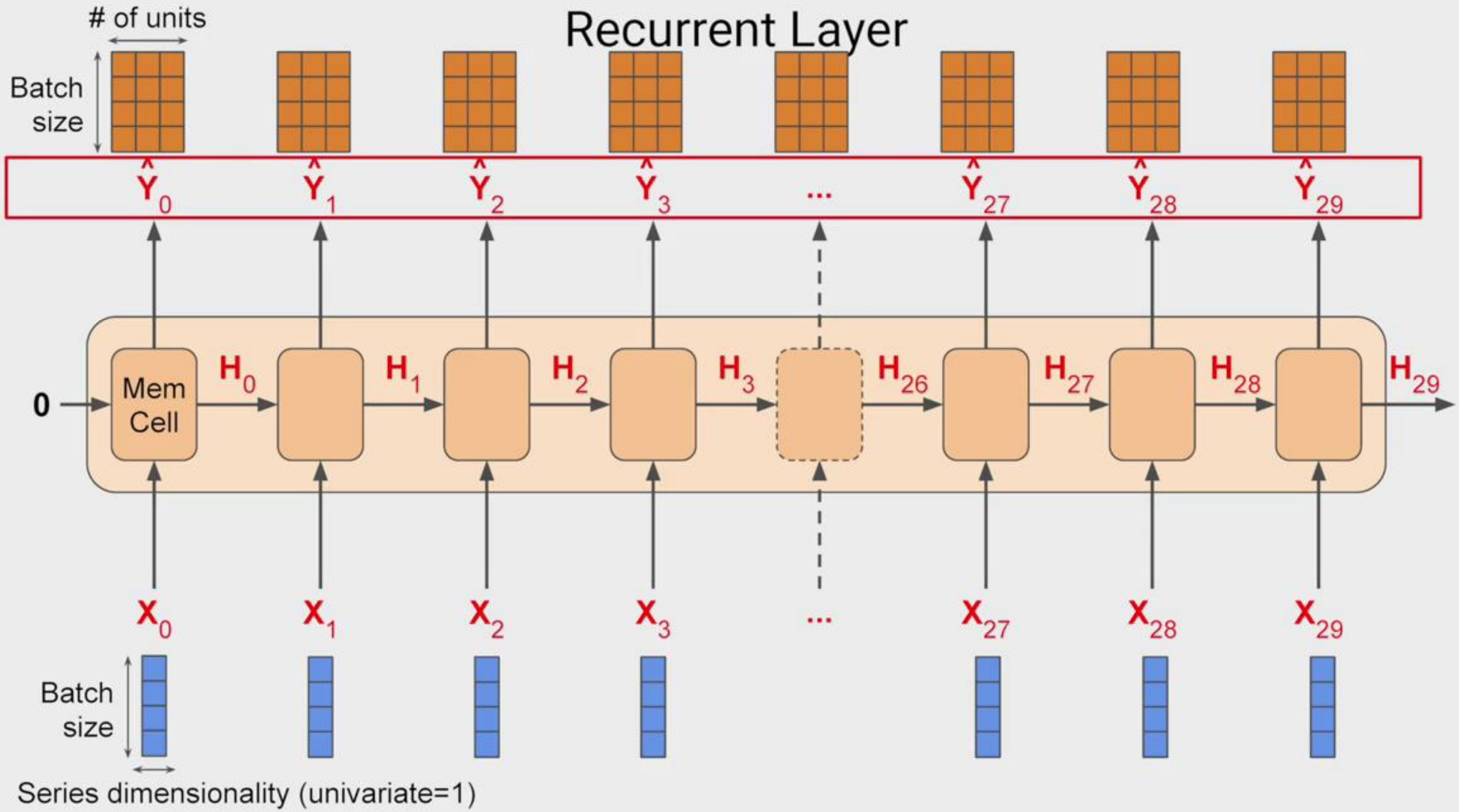
Recurrent Layer

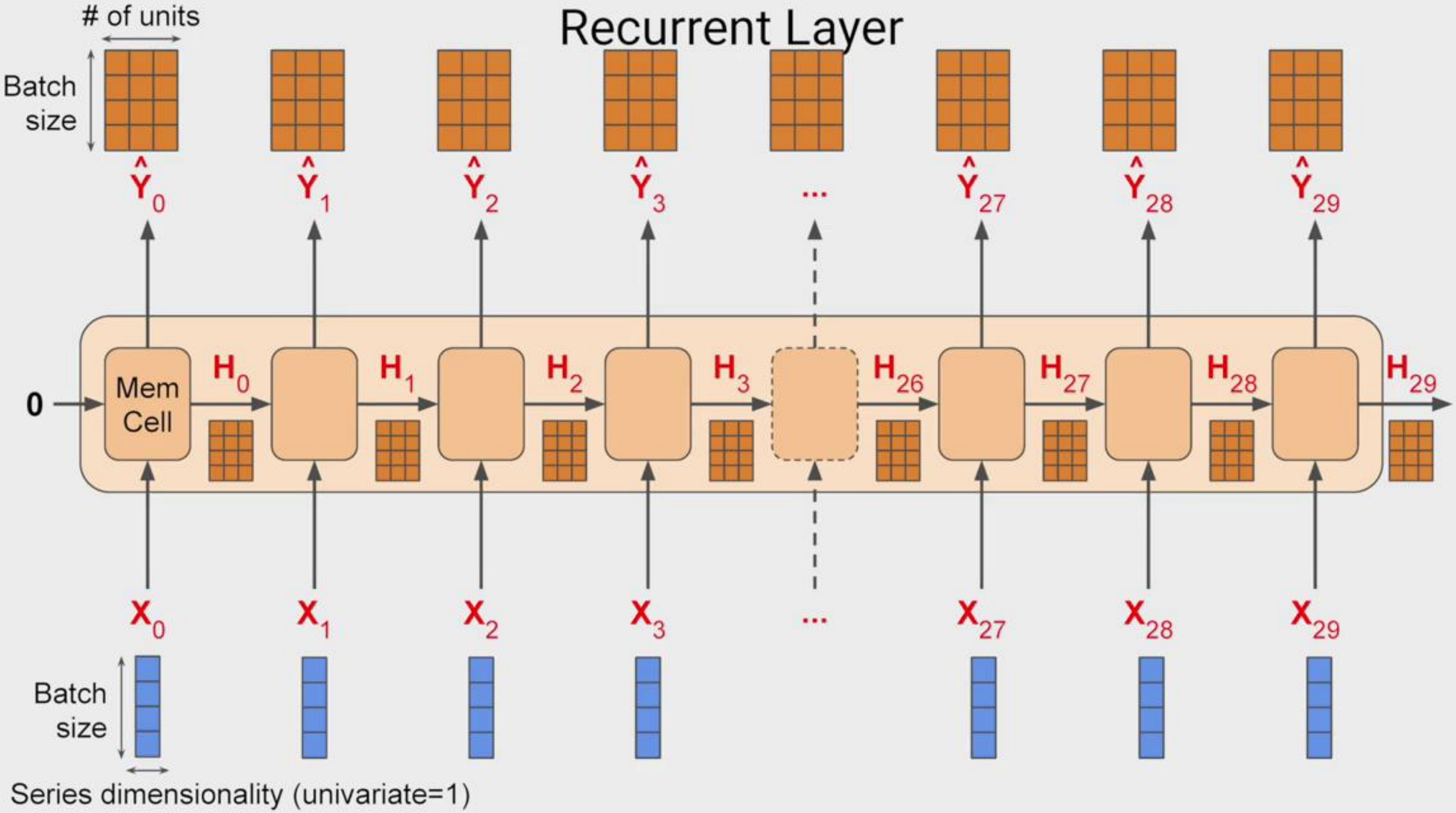


Recurrent Layer

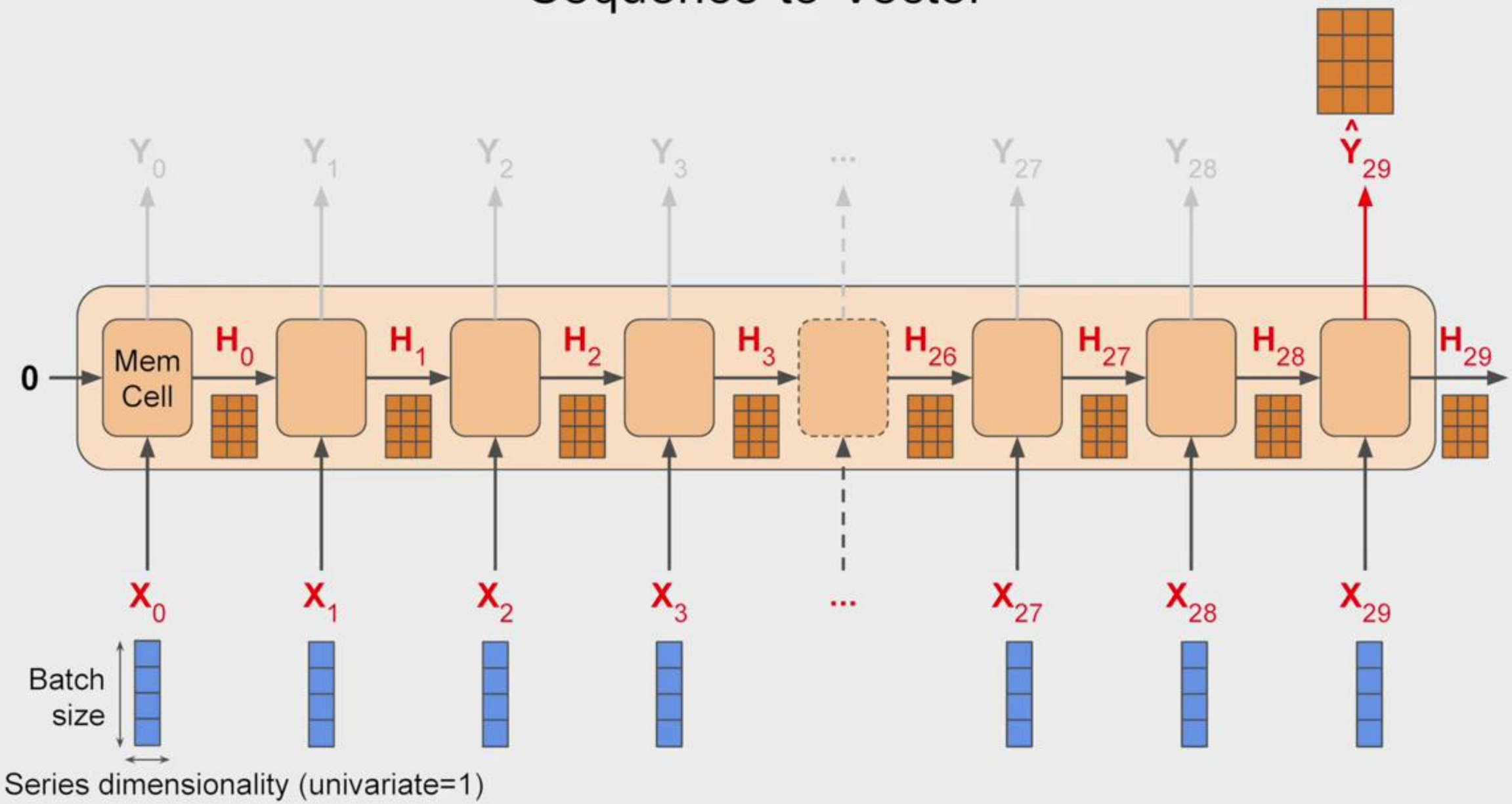



Recurrent Layer
# of units
Batch size
$\hat{Y}_0$
$\hat{Y}_1$
$\hat{Y}_2$
$\hat{Y}_3$
...
$\hat{Y}_{27}$
$\hat{Y}_{28}$
$\hat{Y}_{29}$
0
Mem Cell
H_0
H_1
H_2
H_3
H_{26}
H_{27}
H_{28}
H_{29}
X_0
X_1
X_2
X_3
...
X_{27}
X_{28}
X_{29}
Batch size
Series dimensionality (univariate=1)

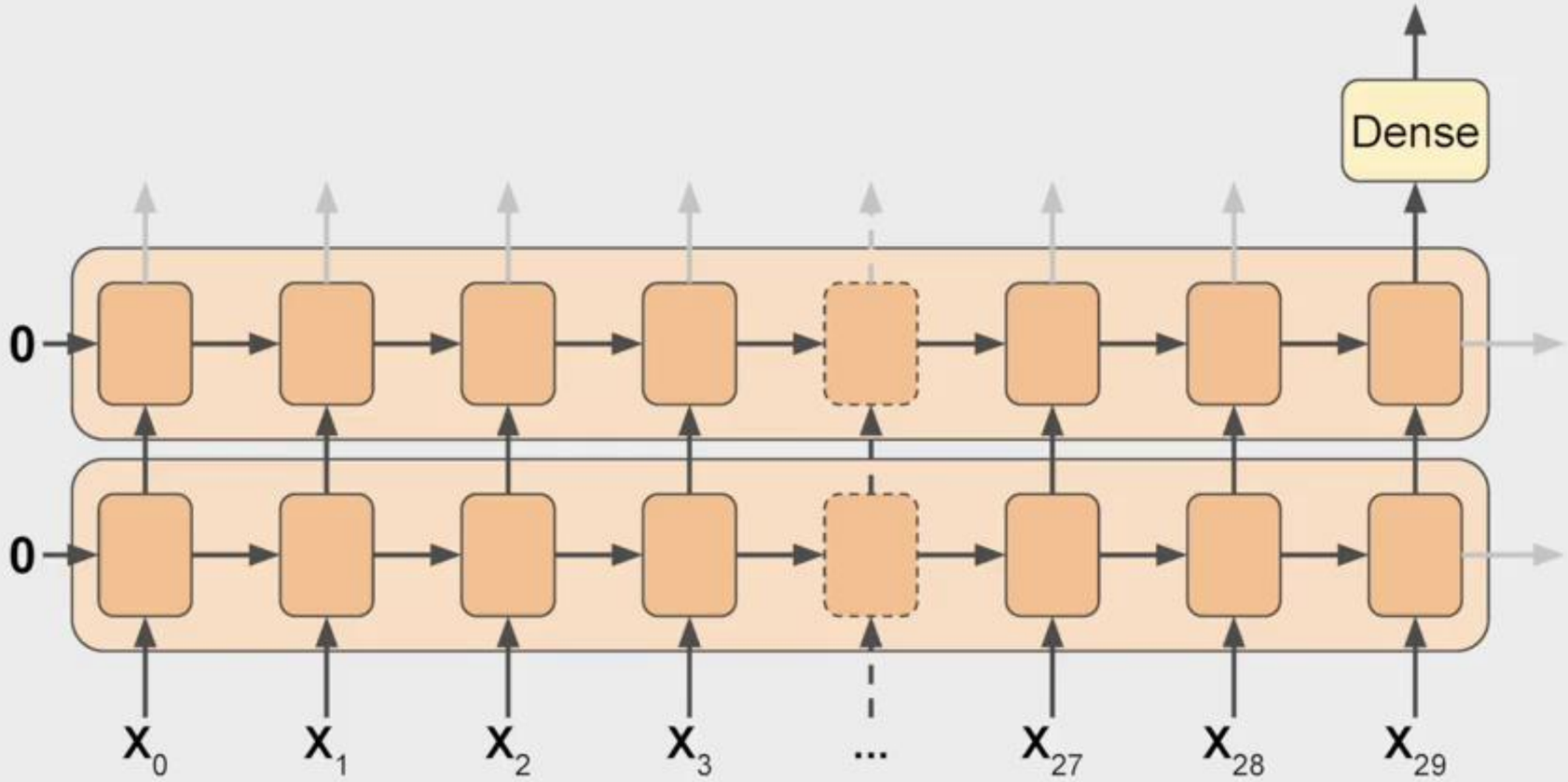

Recurrent Layer
# of units
Batch size
$\hat{Y}_0$ $\hat{Y}_1$ $\hat{Y}_2$ $\hat{Y}_3$ ... $\hat{Y}_{27}$ $\hat{Y}_{28}$ $\hat{Y}_{29}$
0
Mem Cell
H_0 H_1 H_2 H_3 H_{26} H_{27} H_{28} H_{29}
X_0 X_1 X_2 X_3 ... X_{27} X_{28} X_{29}
Batch size
Series dimensionality (univariate=1)


Recurrent Layer
# of units
Batch size
$\hat{Y}_0$
$\hat{Y}_1$
$\hat{Y}_2$
$\hat{Y}_3$
...
$\hat{Y}_{27}$
$\hat{Y}_{28}$
$\hat{Y}_{29}$
0
Mem Cell
H_0
H_1
H_2
H_3
H_{26}
H_{27}
H_{28}
H_{29}
X_0
X_1
X_2
X_3
...
X_{27}
X_{28}
X_{29}
Batch size
Series dimensionality (univariate=1)

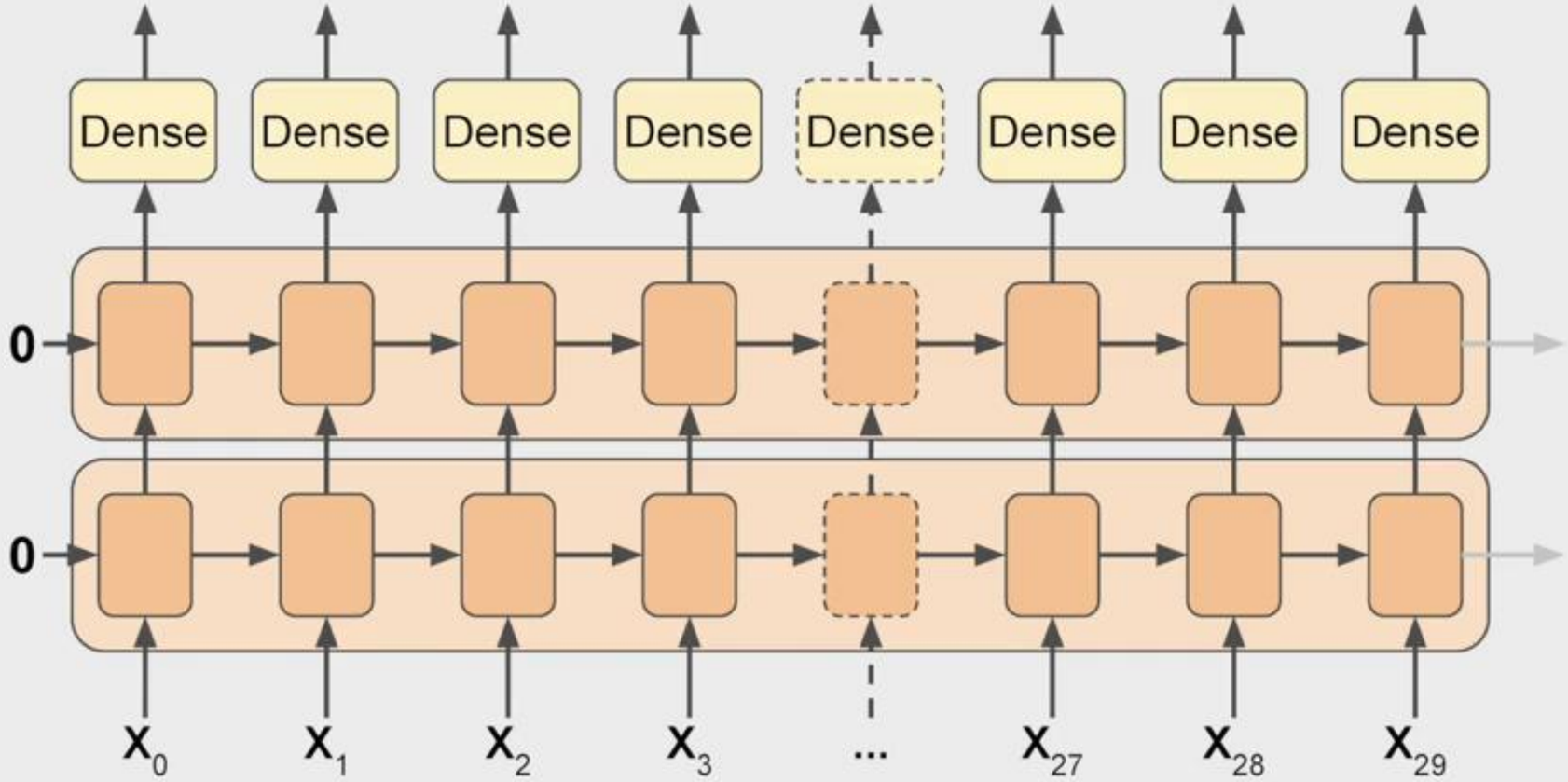
Sequence-to-Vector



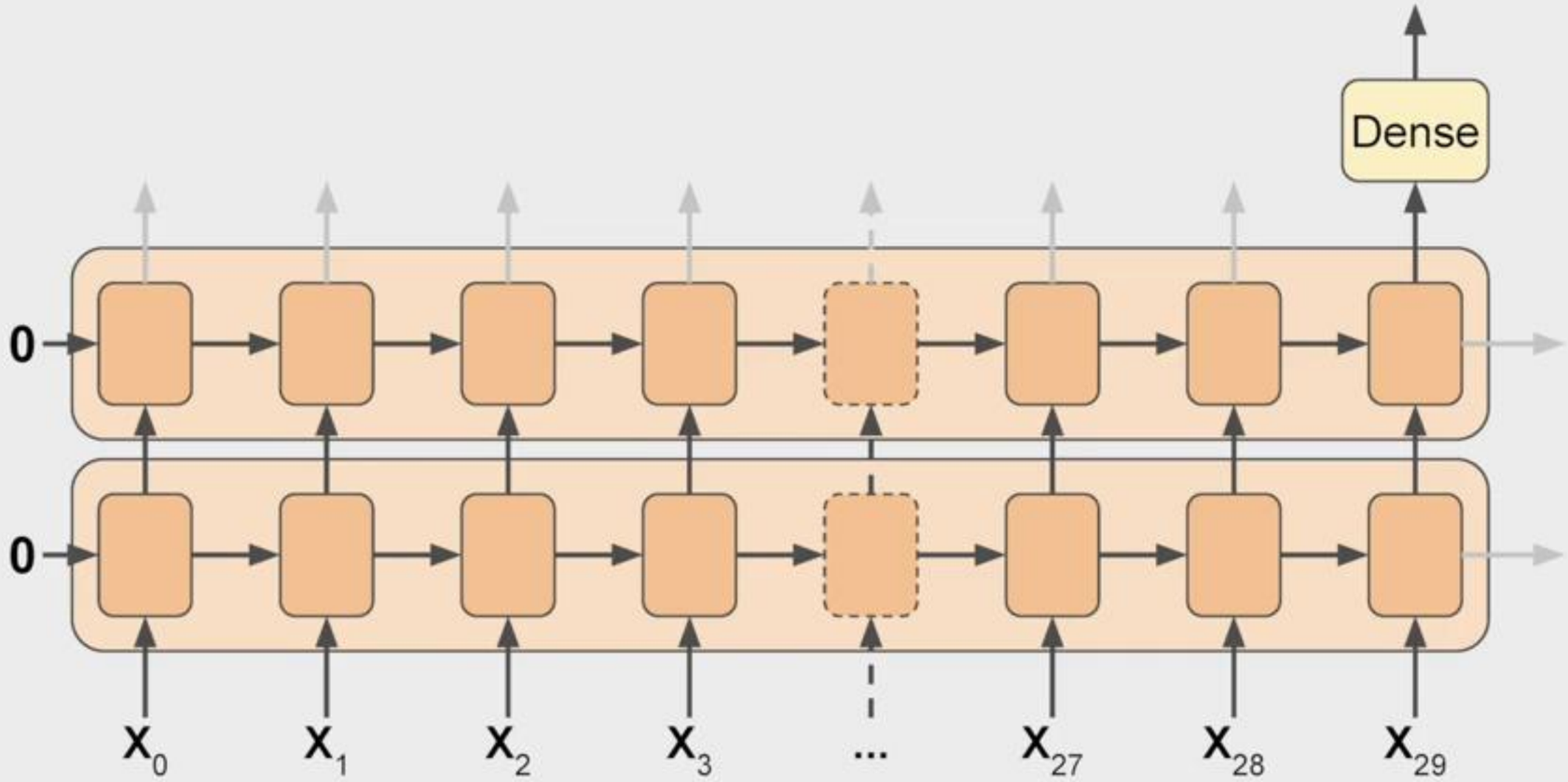
```python
model = keras.models.Sequential([
  keras.layers.SimpleRNN(20, return_sequences=True,
                         input_shape=[None, 1]),
  keras.layers.SimpleRNN(20),
  keras.layers.Dense(1)
])
```



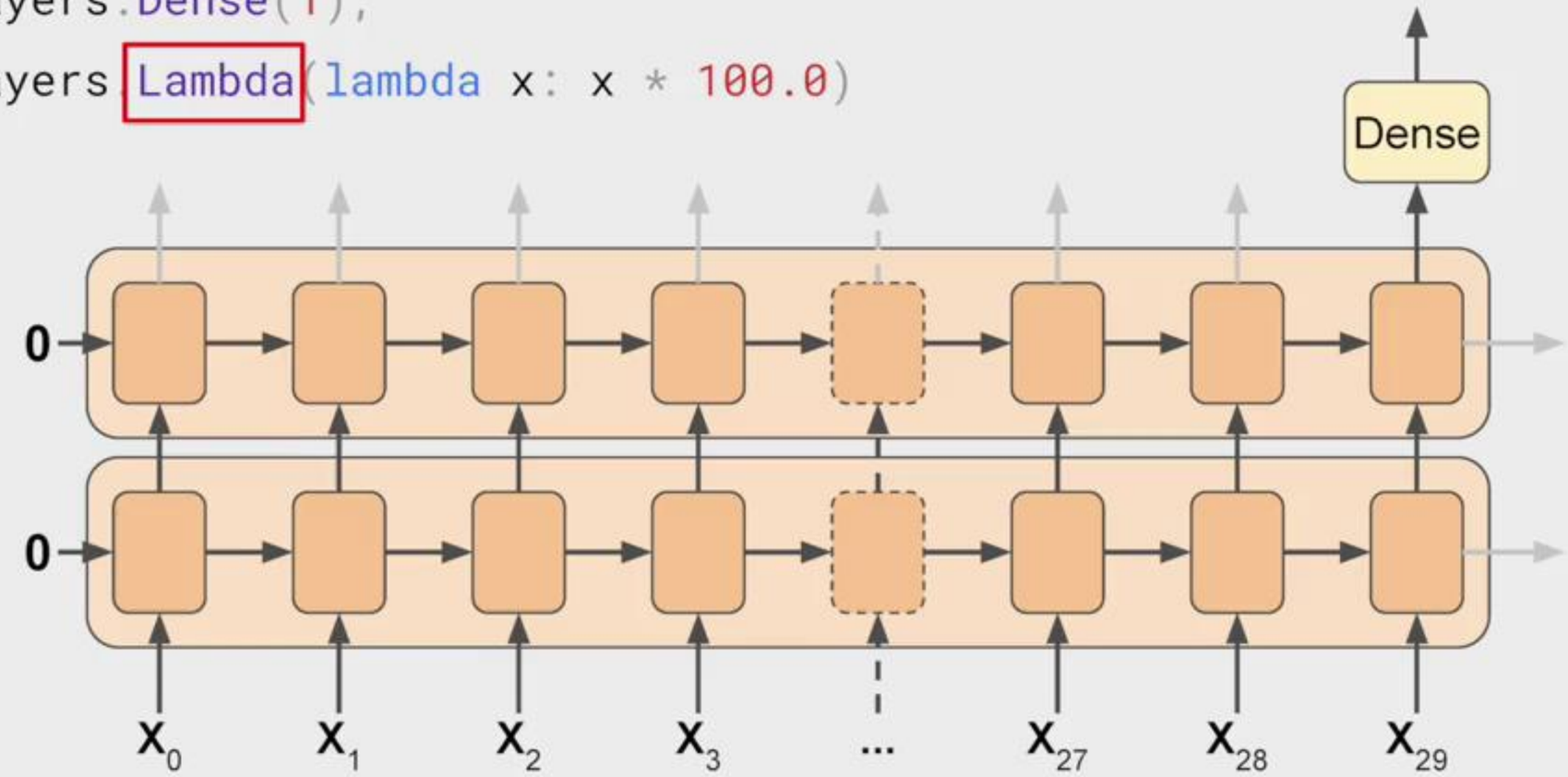
```python
model = keras.models.Sequential([
  keras.layers.SimpleRNN(20, return_sequences=True,
                         input_shape=[None, 1]),
  keras.layers.SimpleRNN(20, return_sequences=True),
  keras.layers.Dense(1)
])
```



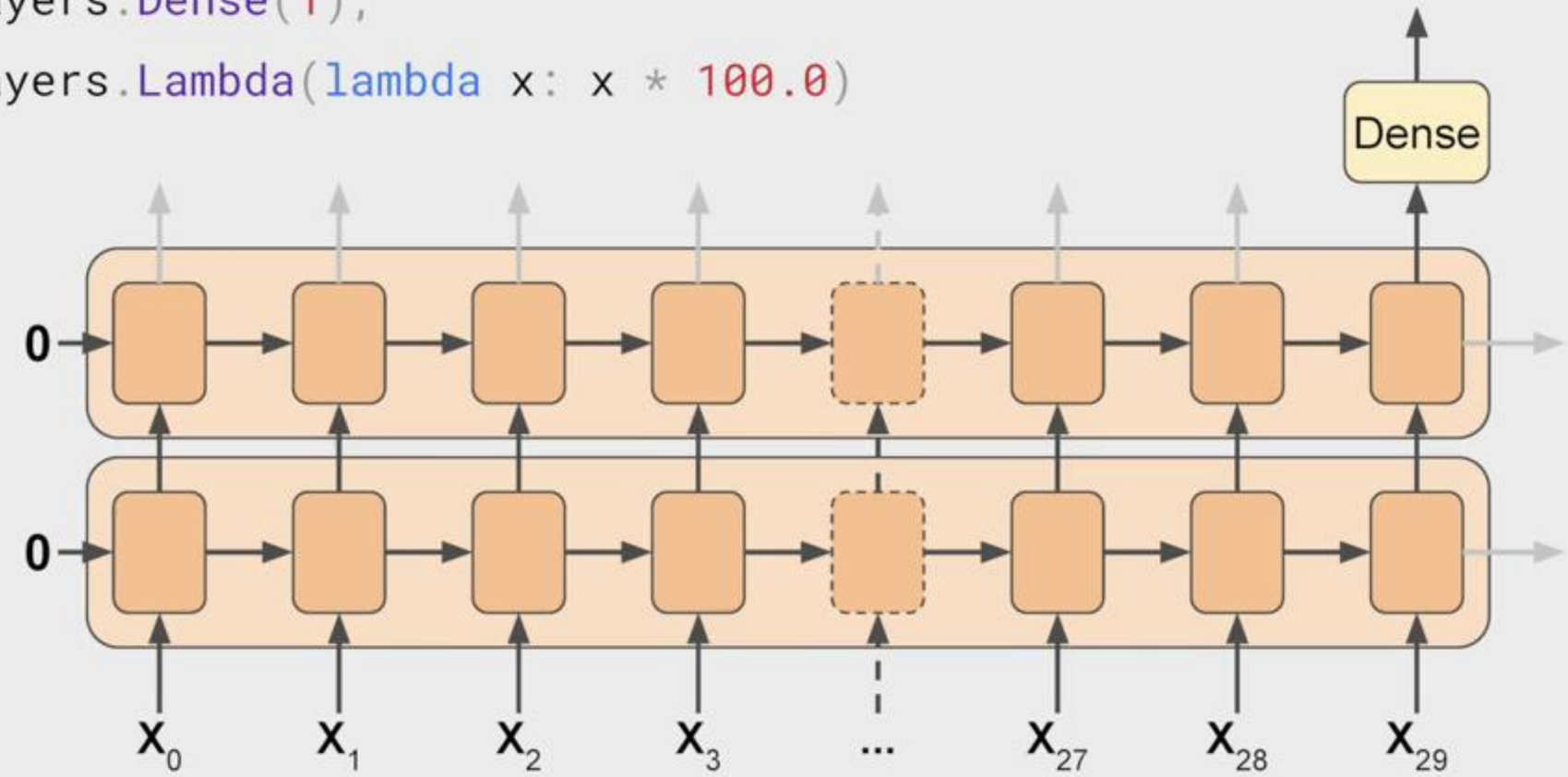
```python
model = keras.models.Sequential([
  keras.layers.SimpleRNN(20, return_sequences=True,
                         input_shape=[None, 1]),
  keras.layers.SimpleRNN(20),
  keras.layers.Dense(1)
])
```



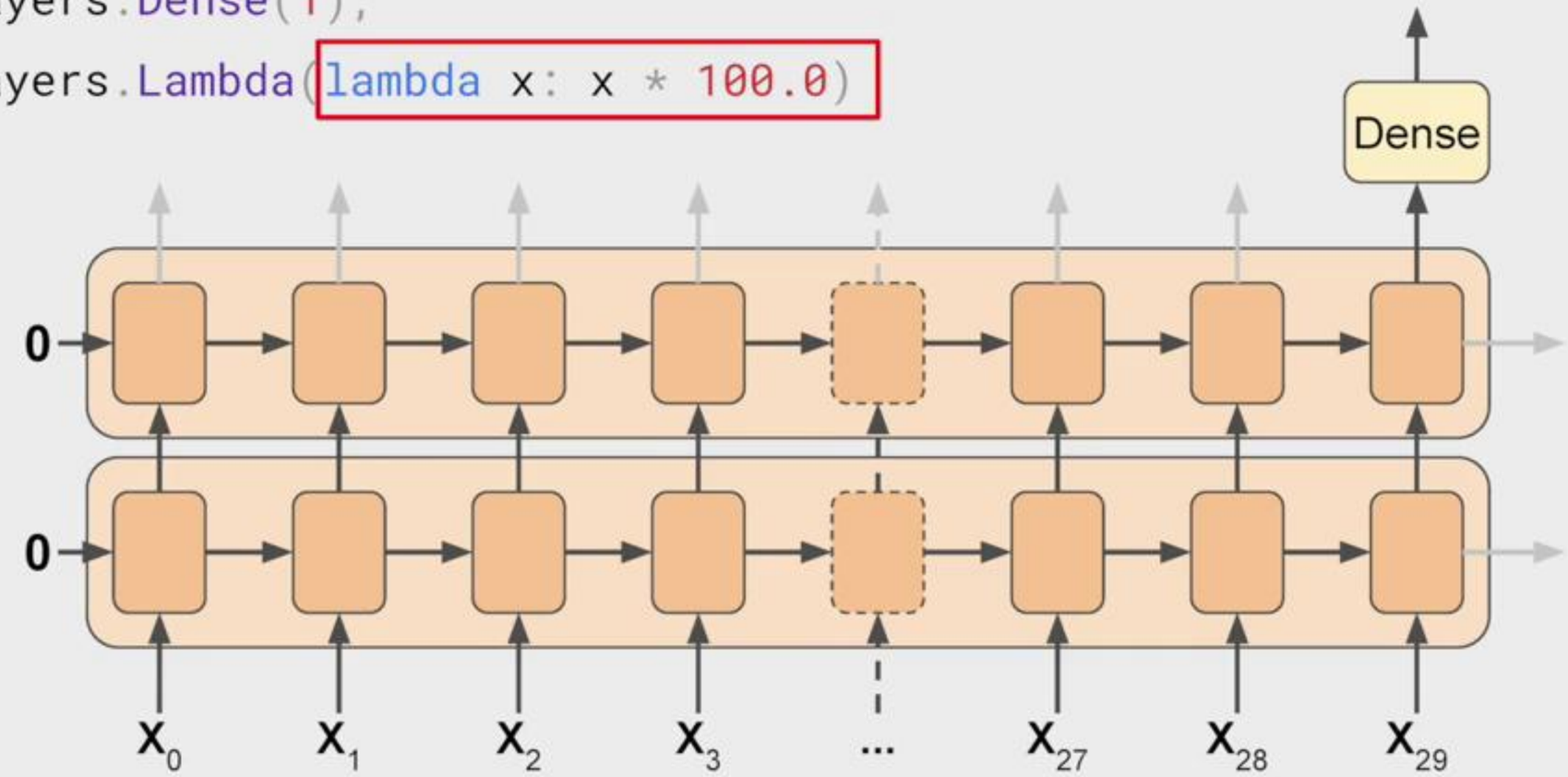
```
model = keras.models.Sequential([
  keras.layers.Lambda(lambda x: tf.expand_dims(x, axis=-1),
                      input_shape=[None]),
  keras.layers.SimpleRNN(20, return_sequences=True),
  keras.layers.SimpleRNN(20),
  keras.layers.Dense(1),
  keras.layers.Lambda(lambda x: x * 100.0)
])
```



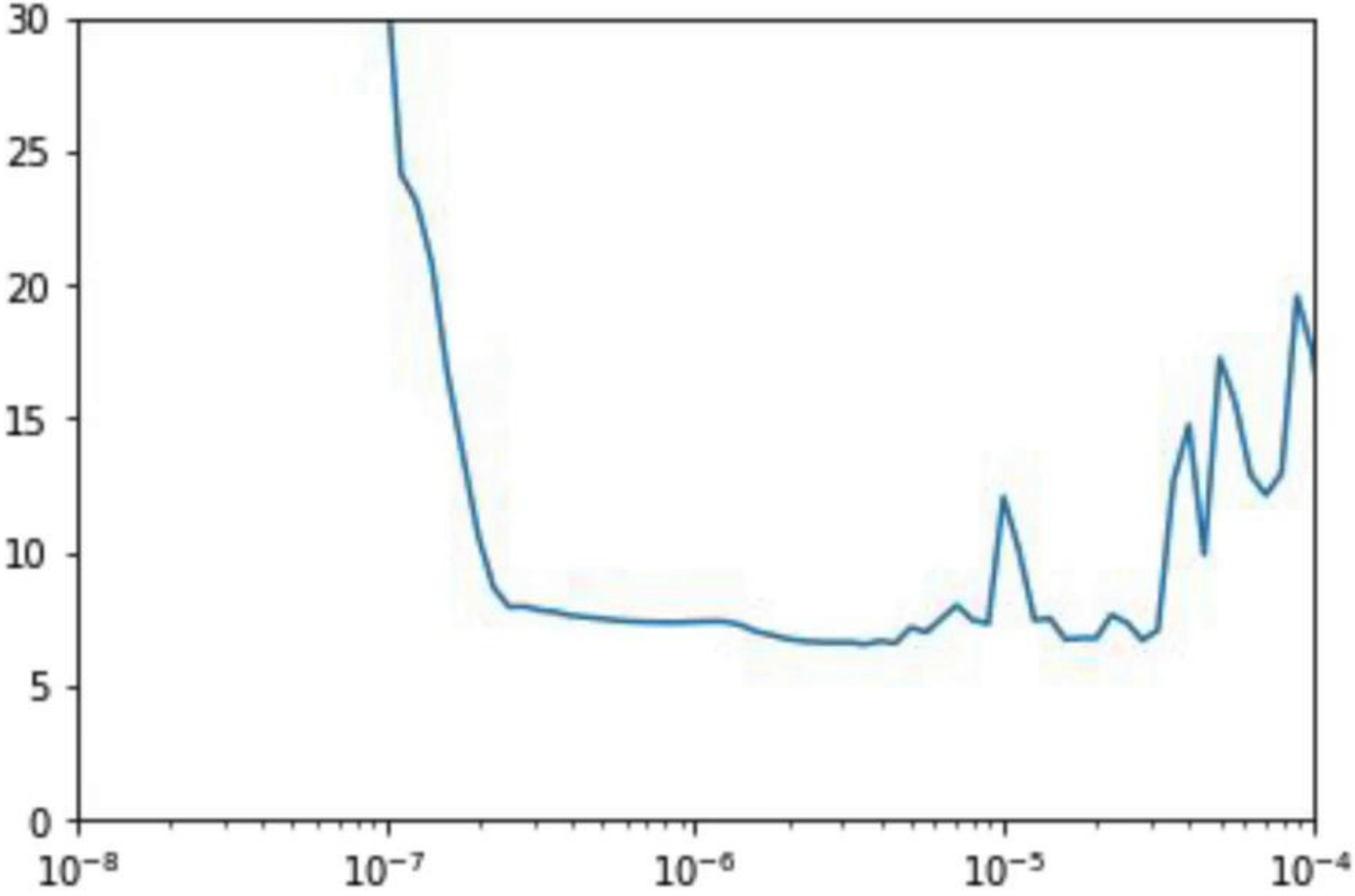
```
model = keras.models.Sequential([
  keras.layers.Lambda(lambda x: tf.expand_dims(x, axis=-1),
                      input_shape=[None]),
  keras.layers.SimpleRNN(20, return_sequences=True),
  keras.layers.SimpleRNN(20),
  keras.layers.Dense(1),
  keras.layers.Lambda(lambda x: x * 100.0)
])
```



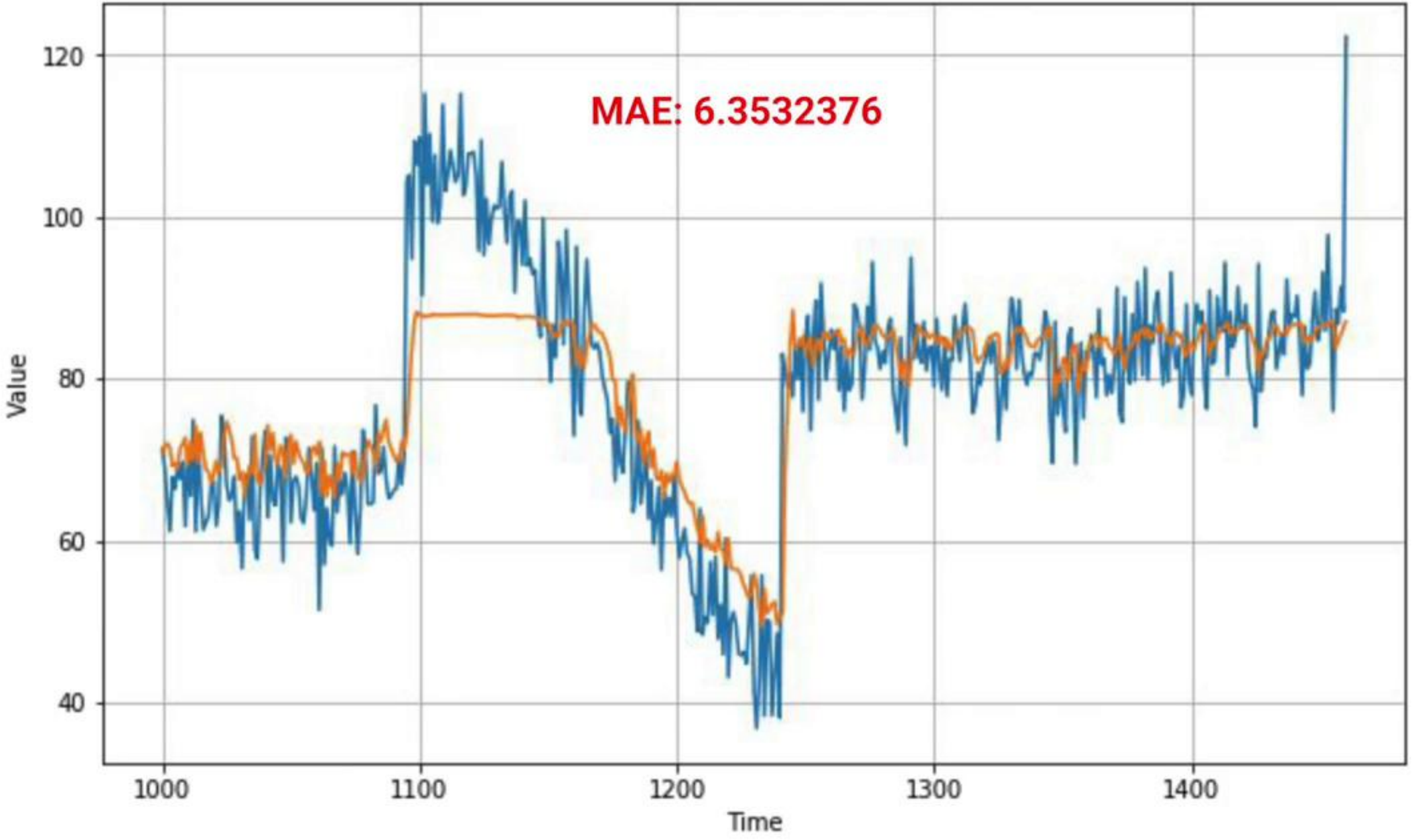
```python
model = keras.models.Sequential([
  keras.layers.Lambda(lambda x: tf.expand_dims(x, axis=-1),
                      input_shape=[None]),
  keras.layers.SimpleRNN(20, return_sequences=True),
  keras.layers.SimpleRNN(20),
  keras.layers.Dense(1),
  keras.layers.Lambda(lambda x: x * 100.0)
])
```

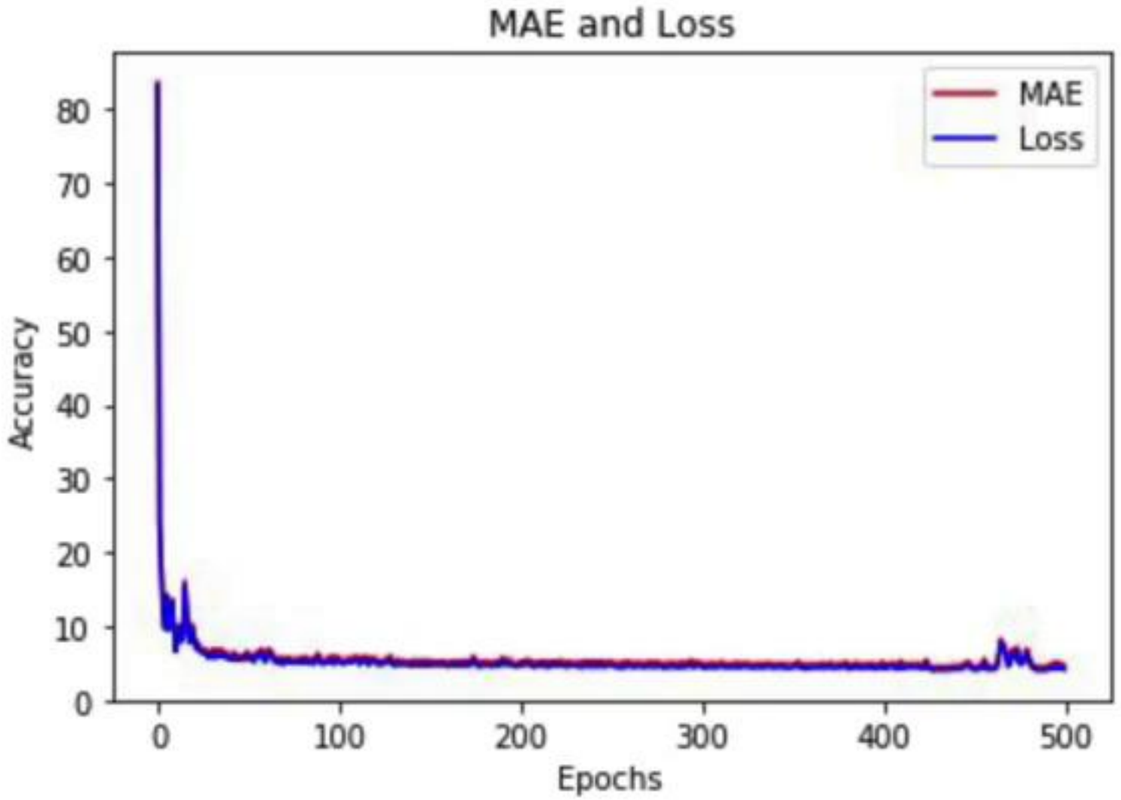


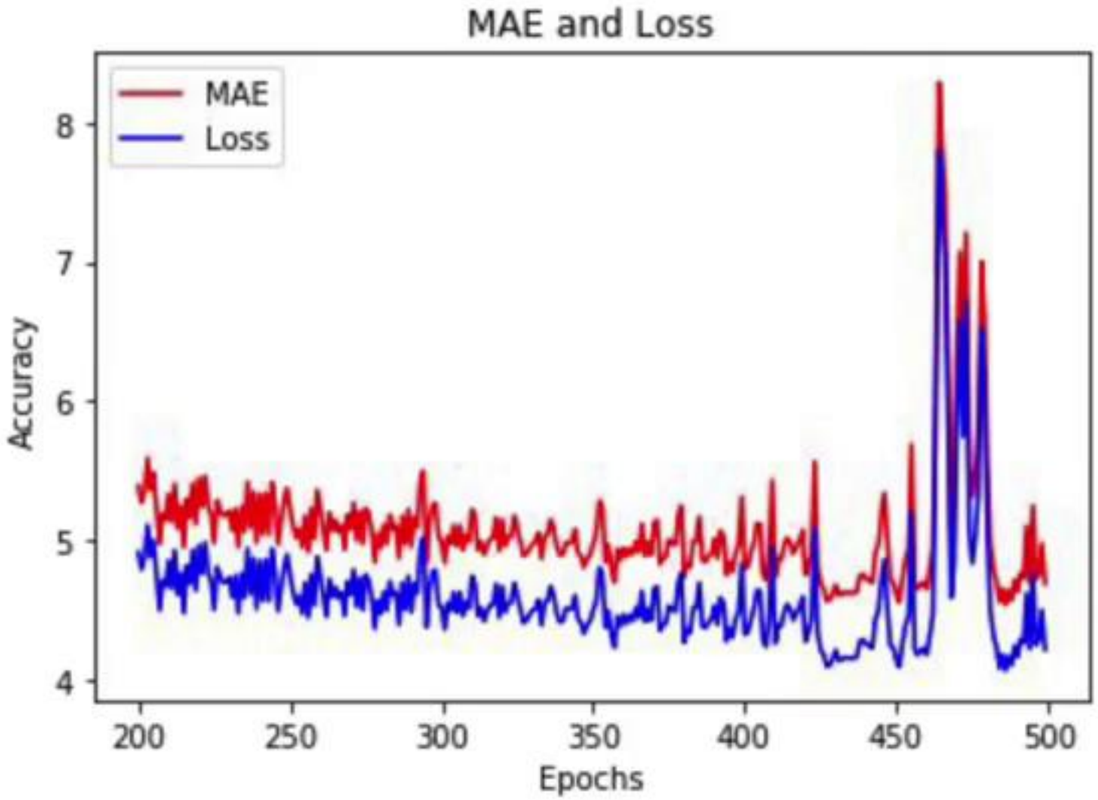
```
train_set = windowed_dataset(x_train, window_size, batch_size=128,
shuffle_buffer=shuffle_buffer_size)

model = tf.keras.models.Sequential([
  tf.keras.layers.Lambda(lambda x: tf.expand_dims(x, axis=-1), input_shape=[None]),
  tf.keras.layers.SimpleRNN(40, return_sequences=True),
  tf.keras.layers.SimpleRNN(40),
  tf.keras.layers.Dense(1),
  tf.keras.layers.Lambda(lambda x: x * 100.0)
])


lr_schedule = tf.keras.callbacks.LearningRateScheduler(lambda epoch: 1e-8 * 10**(epoch / 20))

optimizer = tf.keras.optimizers.SGD(lr=1e-8, momentum=0.9)

model.compile(loss=tf.keras.losses.Huber(),
              optimizer=optimizer,
              metrics=["mae"])

history = model.fit(train_set, epochs=100, callbacks=[lr_schedule])
```

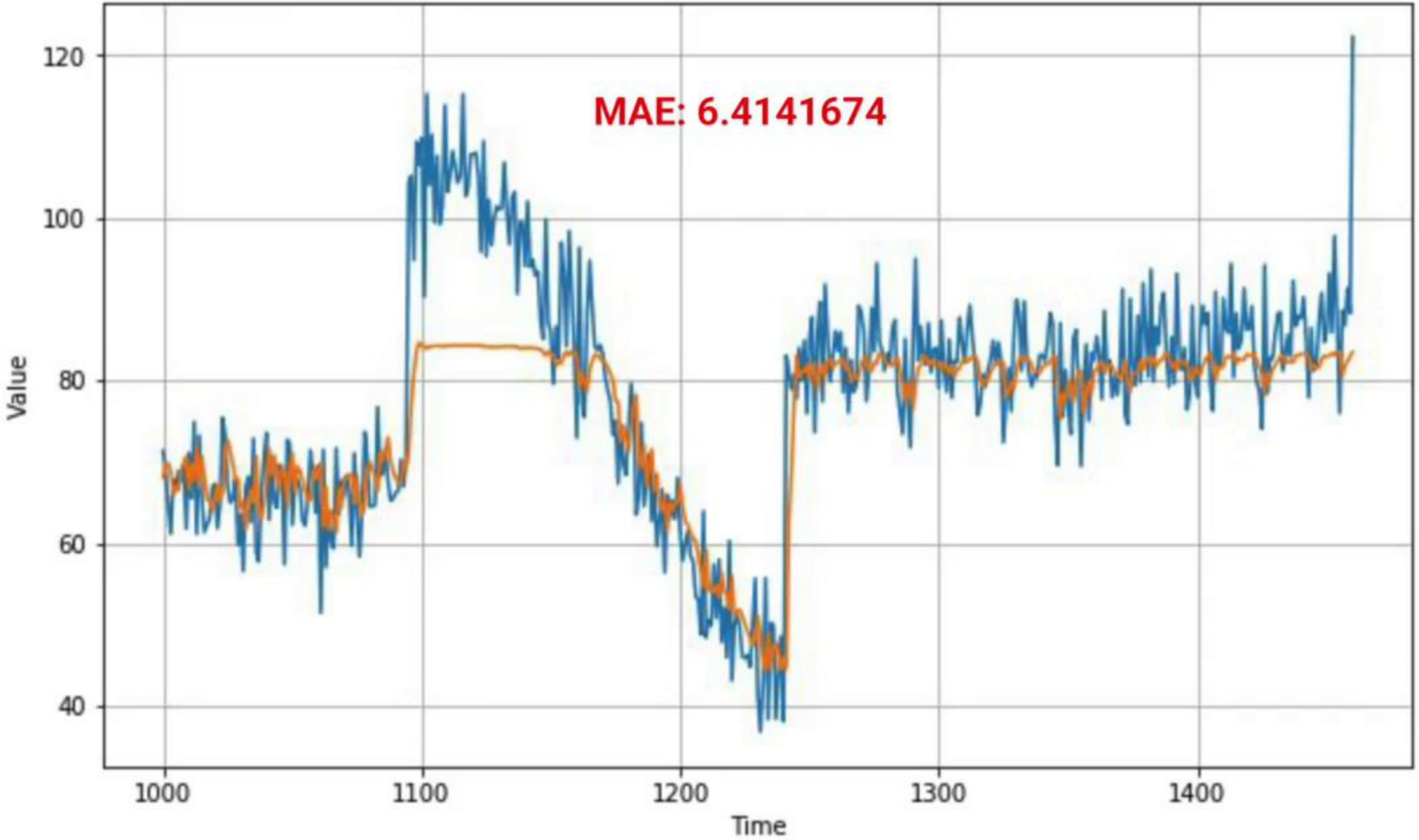

30
25
20
15
10
5
0
10^{-8}
10^{-7}
10^{-6}
10^{-5}
10^{-4}

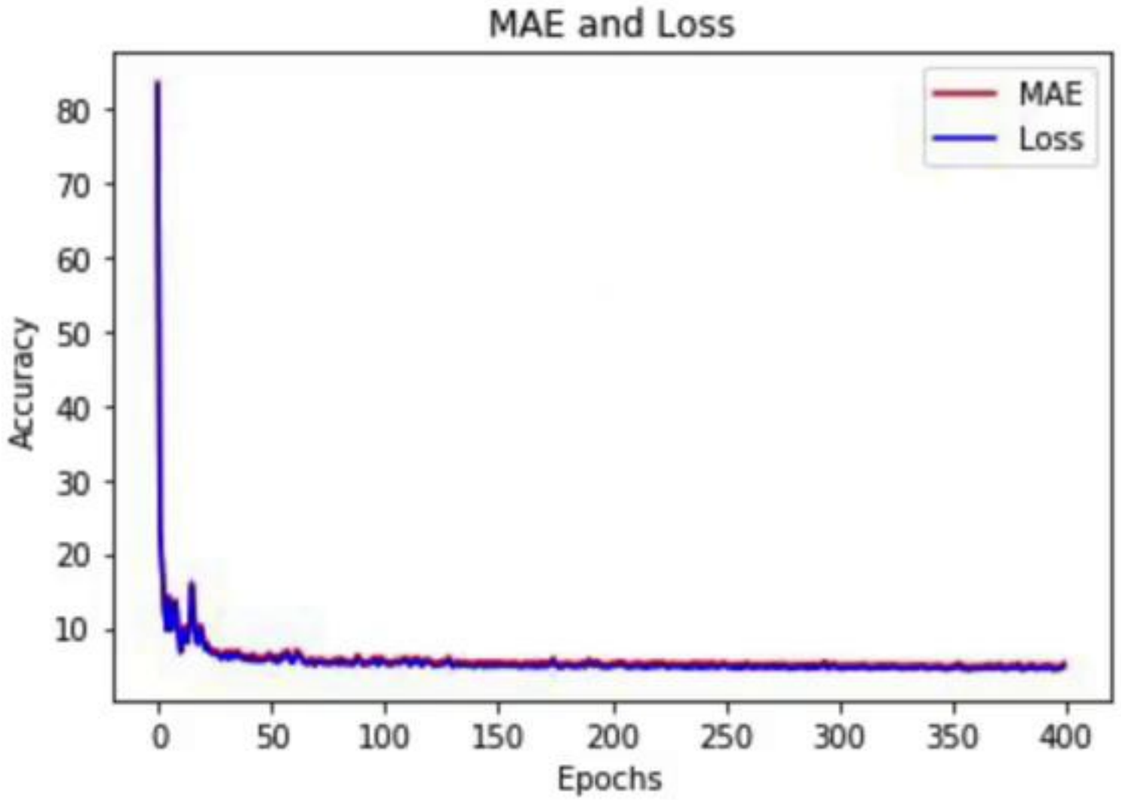
```python
tf.keras.backend.clear_session()
tf.random.set_seed(51)
np.random.seed(51)

dataset = windowed_dataset(x_train, window_size, batch_size=128,
shuffle_buffer=shuffle_buffer_size)

model = tf.keras.models.Sequential([
  tf.keras.layers.Lambda(lambda x: tf.expand_dims(x, axis=-1), input_shape=[None]),
  tf.keras.layers.SimpleRNN(40, return_sequences=True),
  tf.keras.layers.SimpleRNN(40),
  tf.keras.layers.Dense(1),
  tf.keras.layers.Lambda(lambda x: x * 100.0)
])

optimizer = tf.keras.optimizers.SGD(lr=5e-5, momentum=0.9)
history = model.compile(loss=tf.keras.losses.Huber(), optimizer=optimizer, metrics=["mae"])
model.fit(dataset,epochs=500)
```

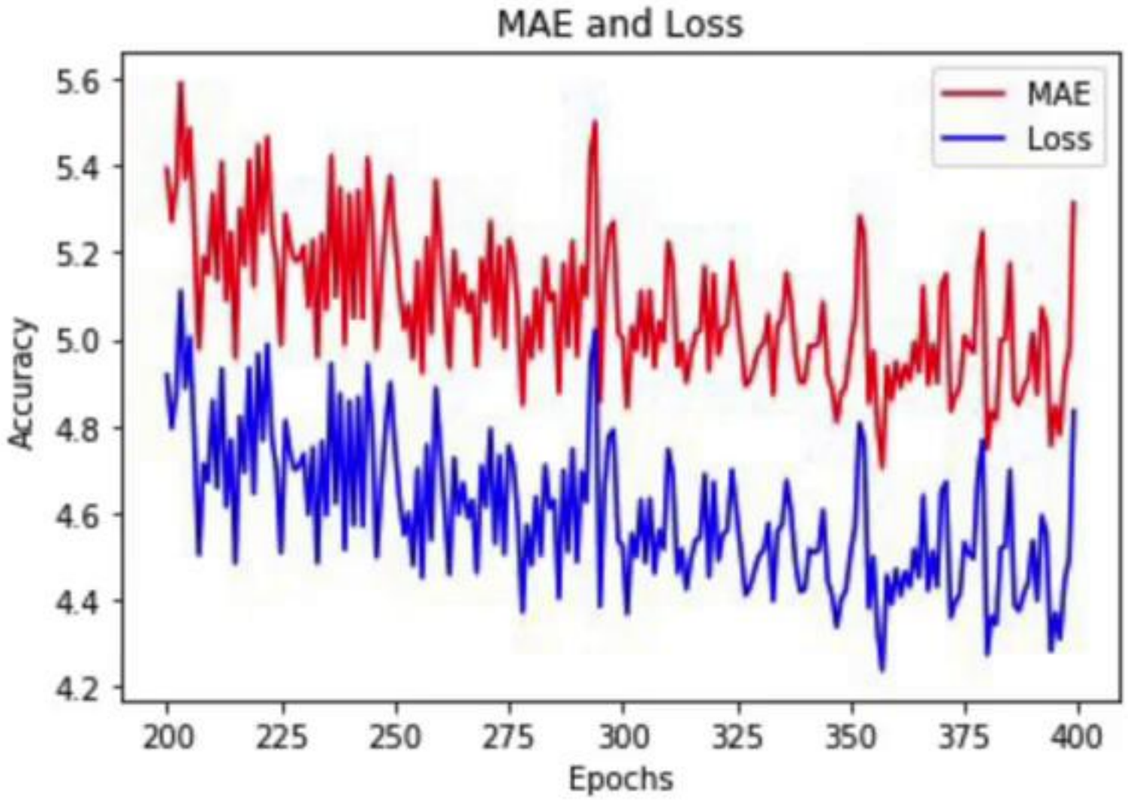

MAE: 6.3532376
120
100
80
60
40
Value
1000
1100
1200
1300
1400
Time

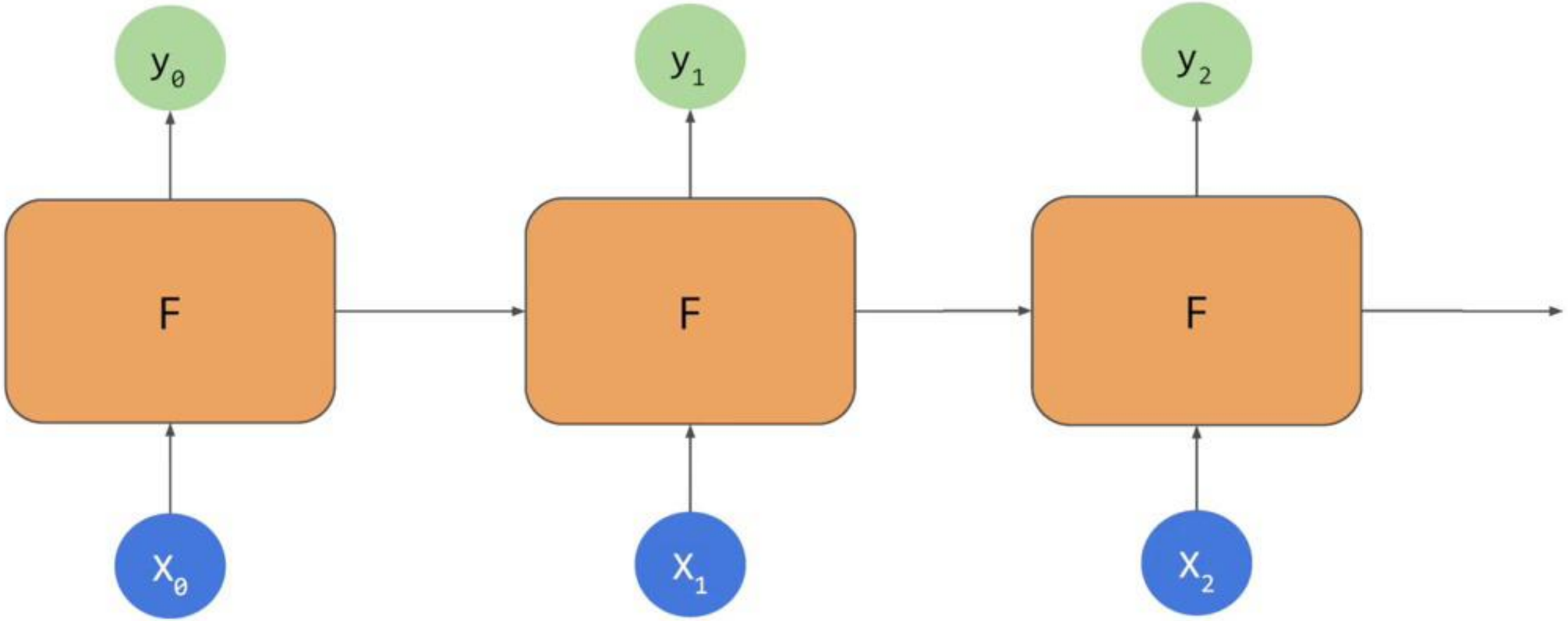

MAE and Loss
MAE
Loss
Accuracy
Epochs
0
10
20
30
40
50
60
70
80
0
100
200
300
400
500

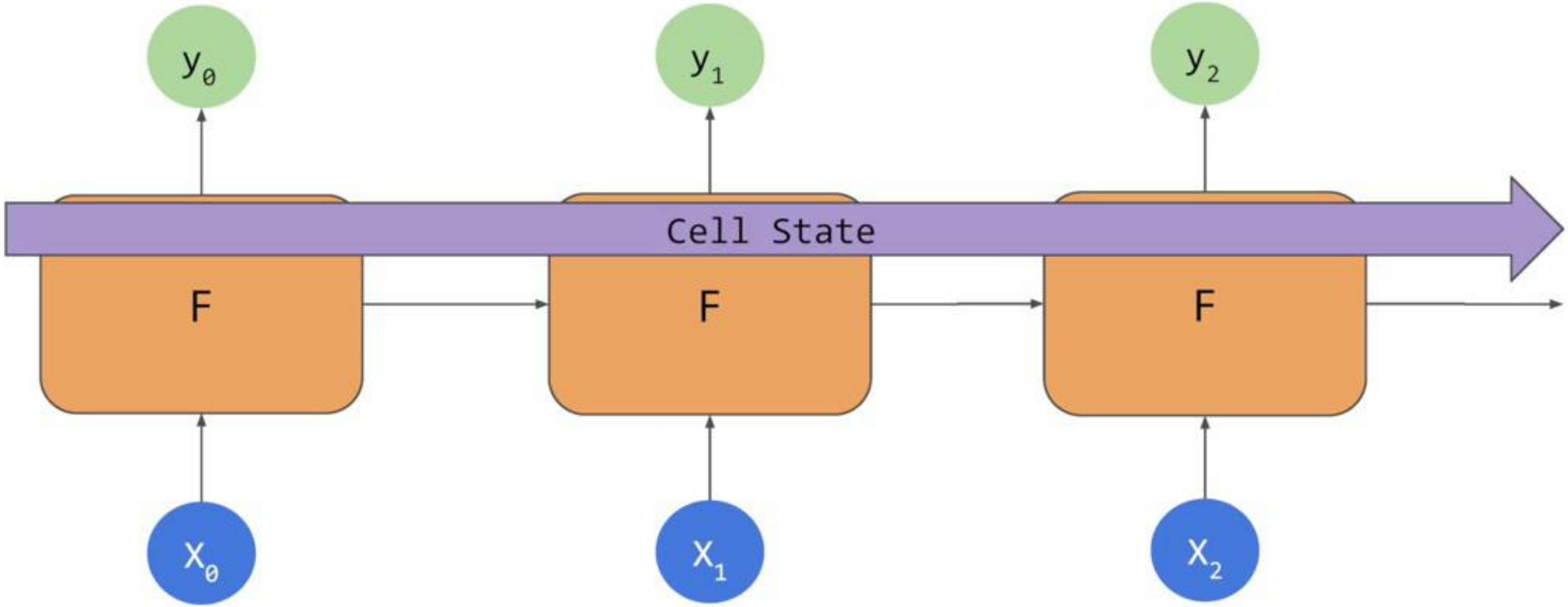

MAE and Loss
MAE
Loss
Accuracy
Epochs
4
5
6
7
8
200
250
300
350
400
450
500

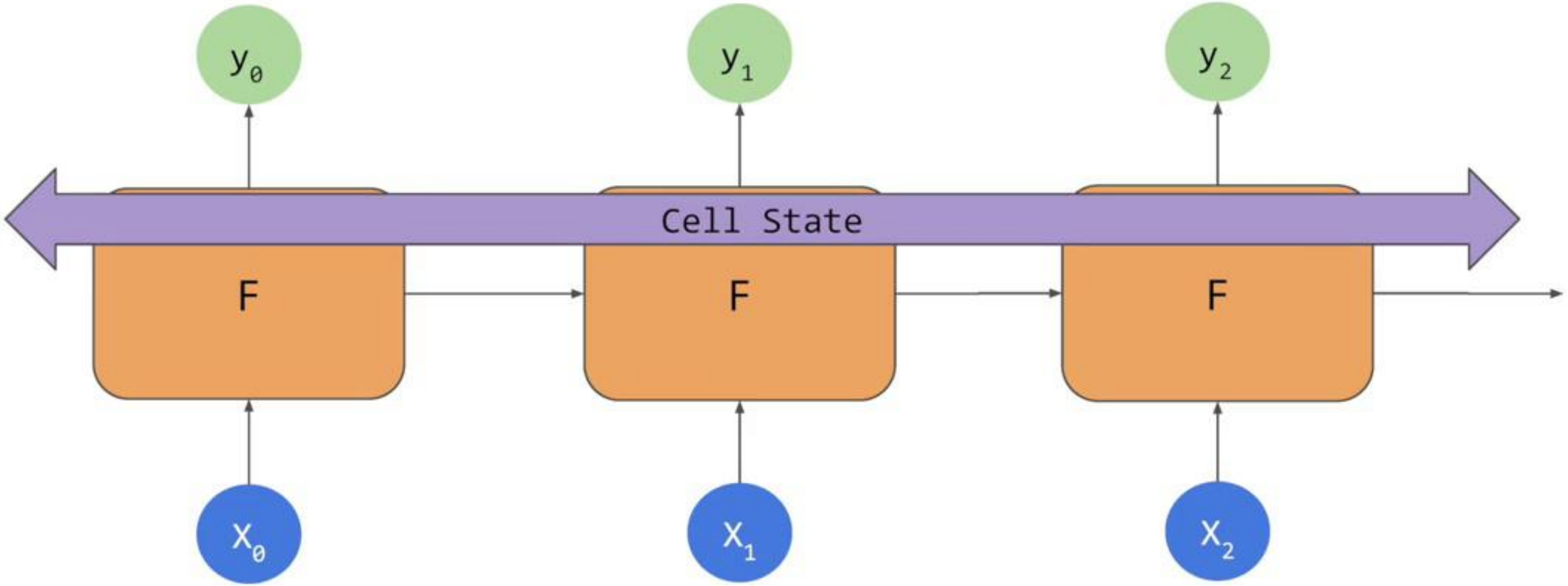

MAE: 6.4141674
120
100
80
60
40
Value
1000
1100
1200
1300
1400
Time


MAE and Loss
MAE
Loss
Accuracy
Epochs
80
70
60
50
40
30
20
10
0
50
100
150
200
250
300
350
400


MAE and Loss
MAE
Loss
Accuracy
Epochs
5.6
5.4
5.2
5.0
4.8
4.6
4.4
4.2
200
225
250
275
300
325
350
375
400

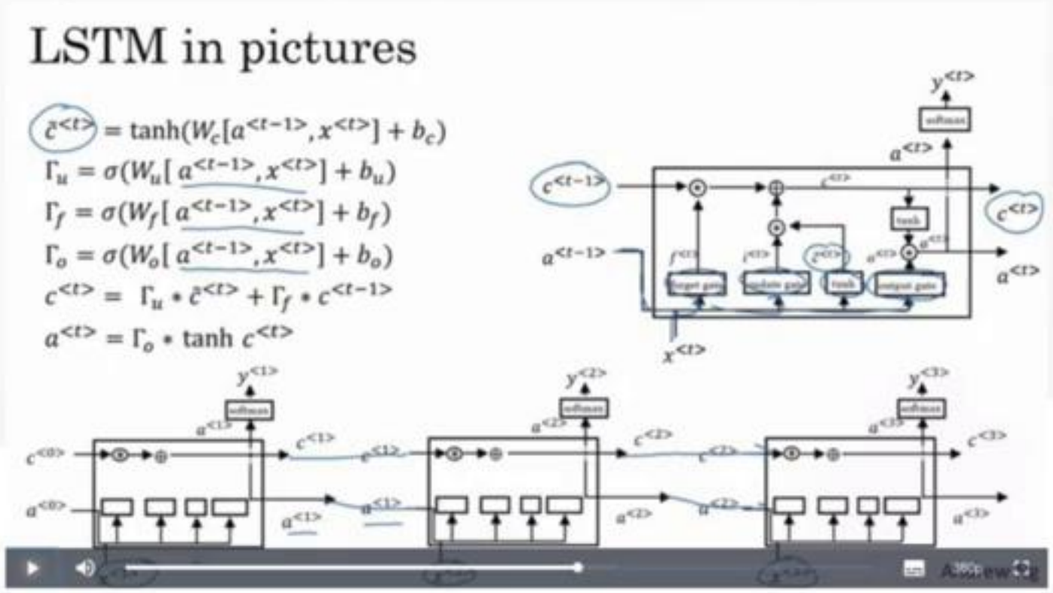

y_0
y_1
y_2
F
F
F
X_0
X_1
X_2


y_0
y_1
y_2
Cell State
F
F
F
X_0
X_1
X_2


y_0
y_1
y_2
Cell State
F
F
F
X_0
X_1
X_2

https://www.coursera.org/lecture/nlp-sequence-models/long-short-term-memory-lstm-KXoay

Long Short Term Memory (LSTM)



From the course by deeplearning.ai

Sequence Models

13393 ratings

Try the Course for Free

This Course | Video Transcript

deeplearning.ai

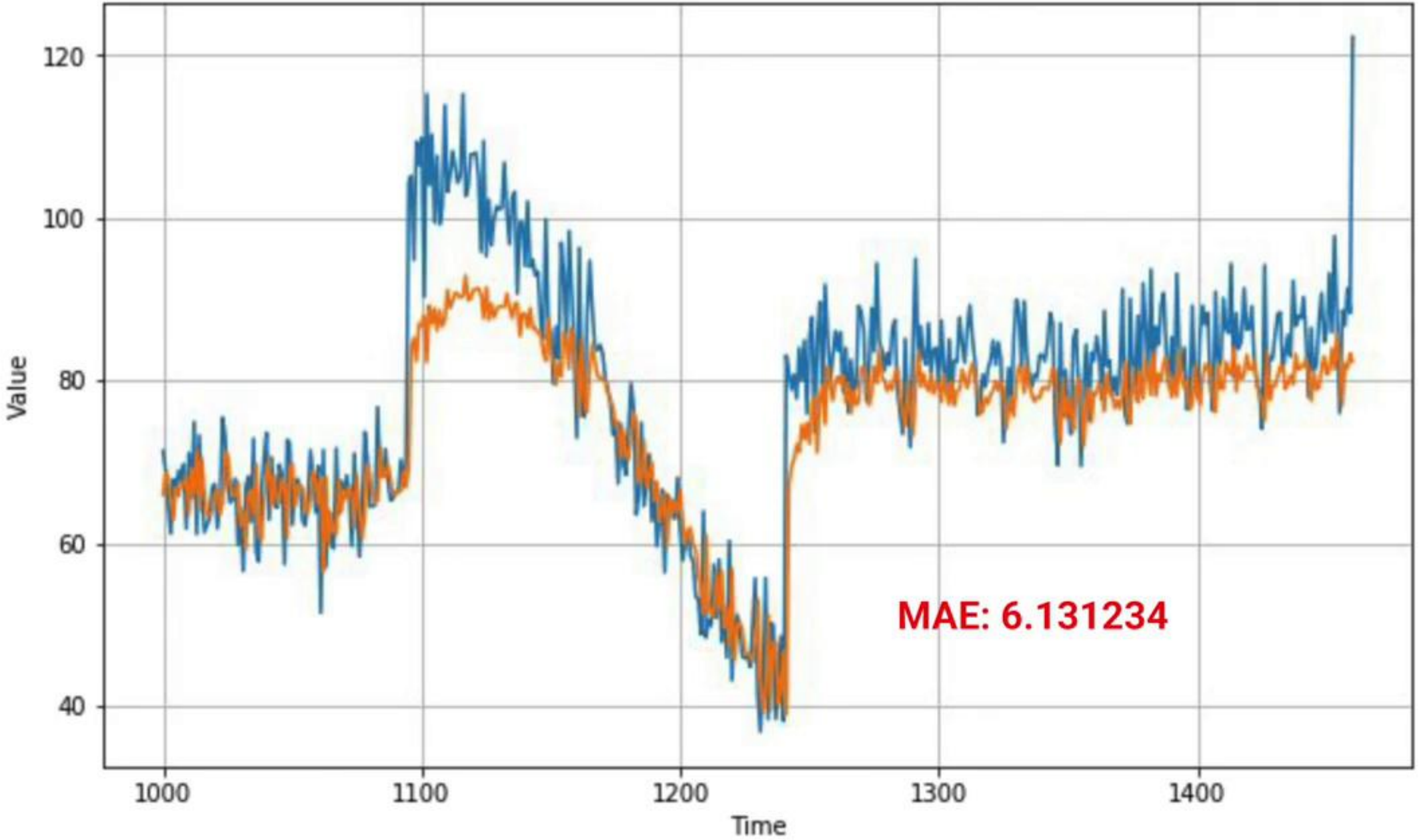
Sequence Models

13393 ratings

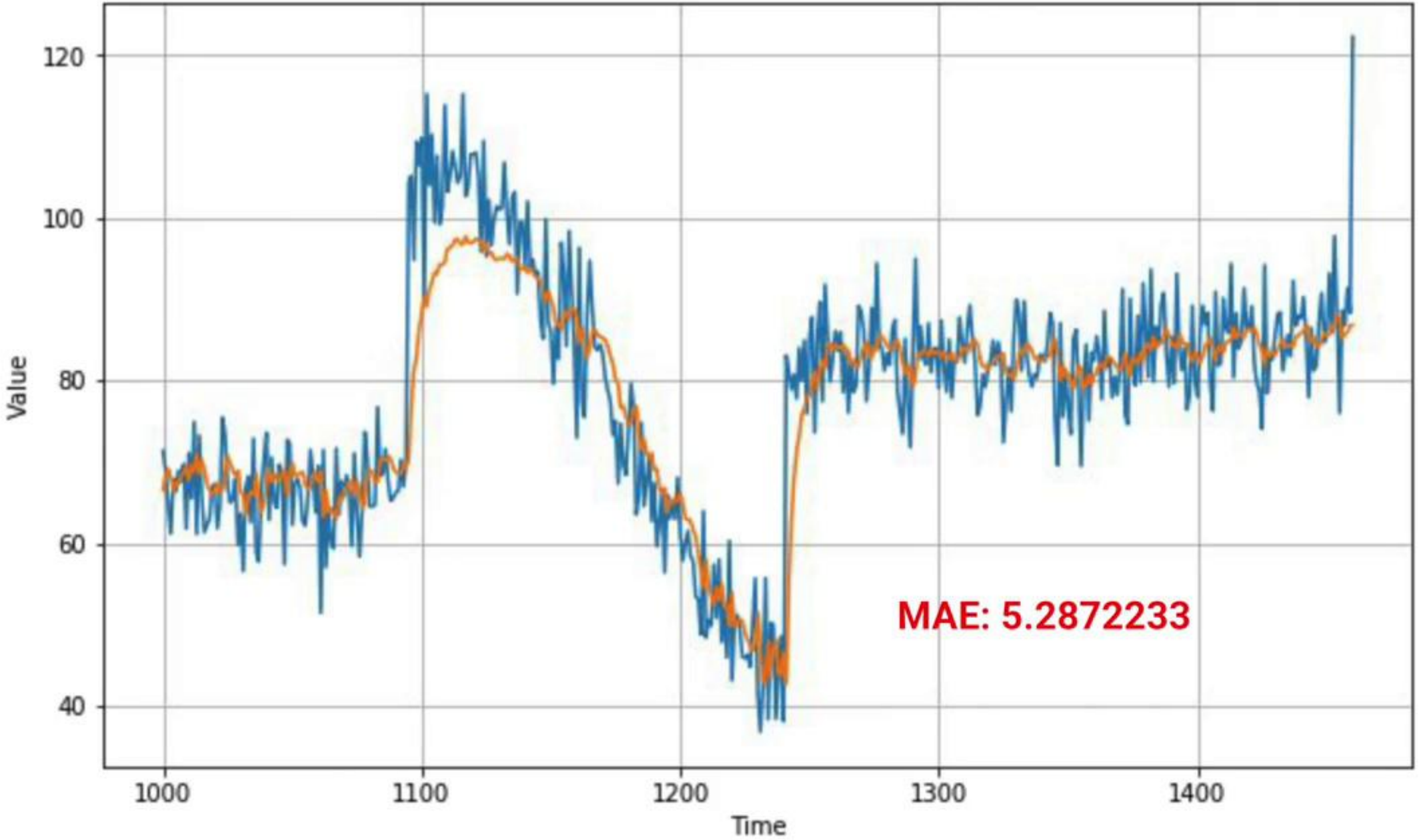
Course 5 of 5 in the Specialization Deep Learning

This course will teach you how to build models for natural language, audio, and other sequence data. Thanks to deep learning, sequence algorithms are working far better than just two years ago, and this is enabling numerous exciting

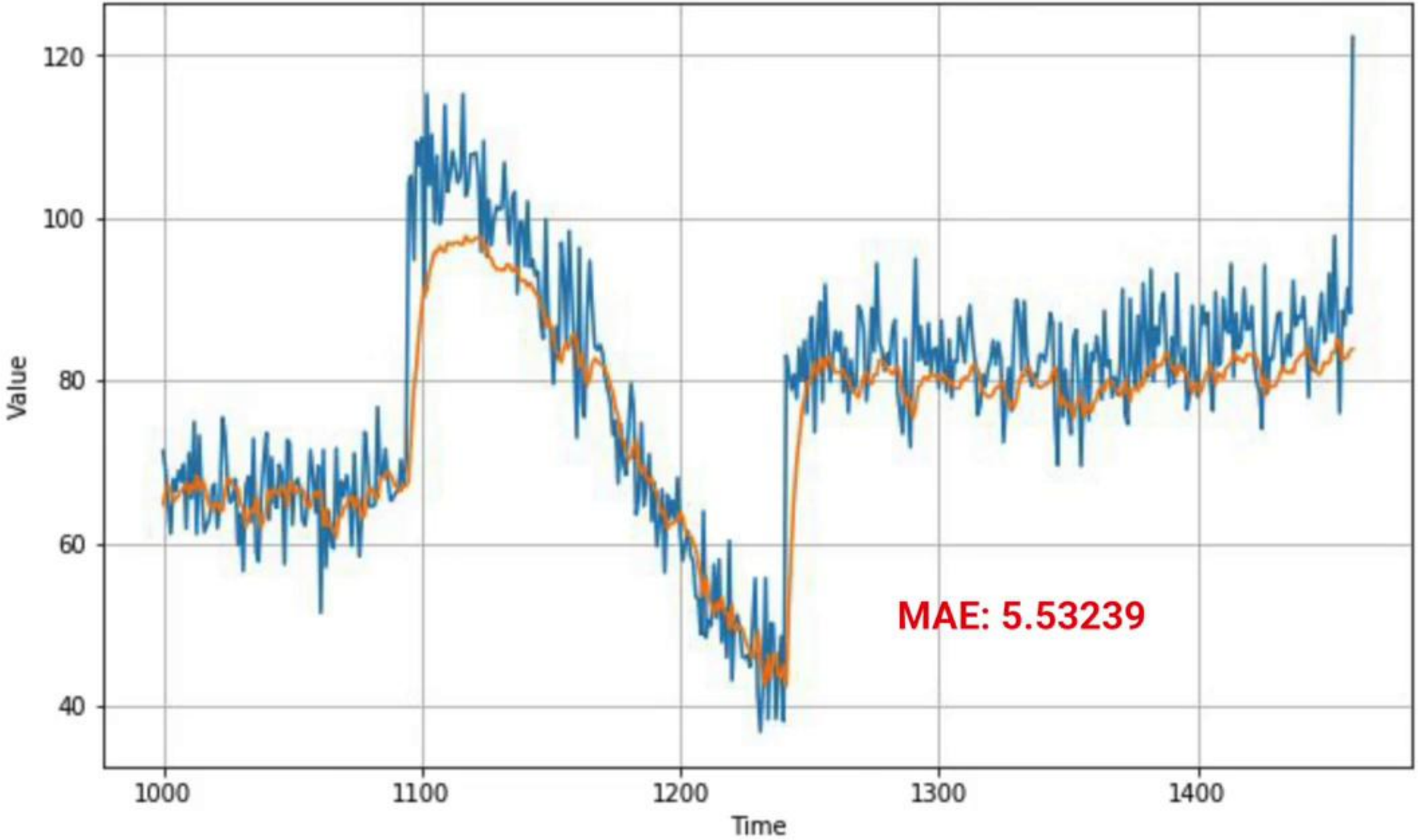
```
tf.keras.backend.clear_session()
dataset = windowed_dataset(x_train, window_size, batch_size, shuffle_buffer_size)

model = tf.keras.models.Sequential([
  tf.keras.layers.Lambda(lambda x: tf.expand_dims(x, axis=-1),
                      input_shape=[None]),
  tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(32)),
  tf.keras.layers.Dense(1),
  tf.keras.layers.Lambda(lambda x: x * 100.0)
])


model.compile(loss="mse", optimizer=tf.keras.optimizers.SGD(lr=1e-6, momentum=0.9))
model.fit(dataset,epochs=100,verbose=0)
```


120
100
80
60
40
Value
1000
1100
1200
1300
1400
Time
MAE: 6.131234

```
tf.keras.backend.clear_session()
dataset = windowed_dataset(x_train, window_size, batch_size, shuffle_buffer_size)

model = tf.keras.models.Sequential([
  tf.keras.layers.Lambda(lambda x: tf.expand_dims(x, axis=-1),
                      input_shape=[None]),
  tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(32, return_sequences=True)),
  tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(32)),
  tf.keras.layers.Dense(1),
  tf.keras.layers.Lambda(lambda x: x * 100.0)
])


model.compile(loss="mse", optimizer=tf.keras.optimizers.SGD(lr=1e-6, momentum=0.9))
model.fit(dataset,epochs=100, verbose=0)
```


120
100
80
60
40
Value
1000
1100
1200
1300
1400
Time
MAE: 5.2872233

```
tf.keras.backend.clear_session()
dataset = windowed_dataset(x_train, window_size, batch_size, shuffle_buffer_size)

model = tf.keras.models.Sequential([
  tf.keras.layers.Lambda(lambda x: tf.expand_dims(x, axis=-1),
                      input_shape=[None]),
  tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(32, return_sequences=True)),
  tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(32, return_sequences=True)),
  tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(32)),
  tf.keras.layers.Dense(1),
  tf.keras.layers.Lambda(lambda x: x * 100.0)
])


model.compile(loss="mse", optimizer=tf.keras.optimizers.SGD(lr=1e-6, momentum=0.9))
model.fit(dataset,epochs=100)
```


120
100
80
60
40
Value
1000
1100
1200
1300
1400
Time
MAE: 5.53239